VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 2: Arrays and Array Operations

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 2 Overview ... 3

Part 1: Array Fundamentals and Fixed-Size Arrays ... 4
- 1.1 The Kinds of Array in SystemVerilog ... 4
- 1.2 Static (Fixed-Size) Arrays ... 6
- 1.3 Packed Arrays and Unpacked Arrays Revisited ... 9
- 1.4 Multidimensional Arrays ... 11
- 1.5 Array Query Functions: $size, $dimensions, $left, $right, $low, $high ... 14

Part 2: Dynamic Arrays and Queues ... 18
- 2.1 Dynamic Arrays ... 18
- 2.2 Queues ... 23

Part 3: Associative Arrays and Choosing an Array Type ... 31
- 3.1 Associative Arrays ... 31
- 3.2 Choosing the Right Array Type ... 39

Part 4: Array Methods: Searching, Ordering and Reduction ... 42
- 4.1 Overview of the Built-in Array Methods ... 42
- 4.2 Array Searching: Locator Methods ... 43
- 4.3 Array Sorting and Ordering Methods ... 48
- 4.4 Array Reduction Methods ... 52

Part 5: Array Initialisation, Assignment, Copying and Comparison ... 56
- 5.1 Array Initialisation ... 56
- 5.2 Array Assignment and Copying ... 57
- 5.3 Array Comparison ... 60
- 5.4 Shallow Copy and Deep Copy ... 62

Part 6: Loops and Array Iteration ... 66
- 6.1 The for Loop ... 66
- 6.2 The foreach Loop ... 68
- 6.3 The repeat Loop ... 70
- 6.4 The while and do-while Loops ... 71
- 6.5 Nested Loops ... 73
- 6.6 Loop Summary and Iterating Safely ... 75

Part 7: Module 2 Revision and Practice ... 77
- 7.1 Revision Summary ... 77
- 7.2 Interview Questions and Answers ... 79
- 7.3 Output-Prediction Questions ... 82
- 7.4 Find-the-Error and Debugging Questions ... 85
- 7.5 Coding Exercises with Solutions ... 89
- 7.6 Concepts You Must Understand Before Moving Forward ... 95
- 7.7 What Comes Next ... 96

Module 2 Overview

Module 1 introduced packed and unpacked arrays as data types. This module covers arrays completely: every kind of array SystemVerilog offers, every built-in method, how arrays are copied and compared, and every loop construct used to walk through them. Arrays are used in every part of a testbench (payloads, scoreboards, memory models, coverage bookkeeping), and array questions appear in almost every SystemVerilog interview, usually as "write code that..." questions.

What this module covers

Part	Content
1	The four kinds of unpacked array, fixed-size (static) arrays, multidimensional arrays, the array query functions (`$size`, `$dimensions`, `$left`, `$right`, `$low`, `$high`)
2	Dynamic arrays (`new[]`, `size()`, `delete()`) and queues (`push_back`, `push_front`, `pop_back`, `pop_front`, `insert`, `delete`)
3	Associative arrays (`exists`, `first`, `last`, `next`, `prev`, `num`, `delete`) and how to choose between the array kinds
4	Array methods: searching (`find` and its relatives, `min`, `max`, `unique`), ordering (`sort`, `rsort`, `reverse`, `shuffle`), reduction (`sum`, `product`, `and`, `or`, `xor`)
5	Assignment, initialisation, copying, comparison, shallow and deep copy
6	Loops: `for`, `foreach`, `repeat`, `while`, `do ... while`, `forever`, nested loops, `break` and `continue`
7	Revision, interview questions, output-prediction and debugging questions, coding exercises

What you need from Module 1

- Packed versus unpacked dimensions and the index order rule (Sections 3.1 and 3.2).
- The difference between `{}` (concatenation) and `'{}` (assignment pattern).
- Two-state and four-state default values (0 and X).
- Expression width rules (Section 4.4), which explain the most common bug with `sum()`.
- `typedef`, `struct` and `enum`.

Conventions

The naming conventions of Module 1 continue: `tb_<topic>` for demonstration modules, `exNN_<name>` for exercise solutions, `_t` / `_e` type suffixes, and the 8-bit ALU (`alu_pkg`, `alu_op_e`, `alu_txn`) as the running example whenever a verification context is needed. A few examples use small classes or functions before Modules 3 and 4 explain them fully; each such use is explained where it appears.

> **Note:** As in Module 1, the "Expected output" blocks were derived by hand from the IEEE 1800 rules, not captured from a simulator. The text printed by the `%p` format specifier (used here to print whole arrays) varies slightly between simulators in spacing and number format; the values are the same. Where an output genuinely depends on the simulator or on a random seed, the text says so.

Part 1: Array Fundamentals and Fixed-Size Arrays

1.1 The Kinds of Array in SystemVerilog

In plain English

An array is a numbered collection of items of the same type, stored under one name. Instead of declaring `data0`, `data1`, ... `data99`, you declare `data [100]` and select an element with an index: `data[37]`.

Different jobs need different kinds of collection:

- A register file with exactly 32 registers needs a collection whose size never changes.
- A packet payload whose length is chosen at random for each packet needs a collection whose size is decided while the simulation runs.
- A list of expected results in a scoreboard needs a collection that grows at one end and shrinks at the other, continuously.
- A model of a 4 GB memory, of which the test touches 50 locations, needs a collection that stores only the locations actually used.

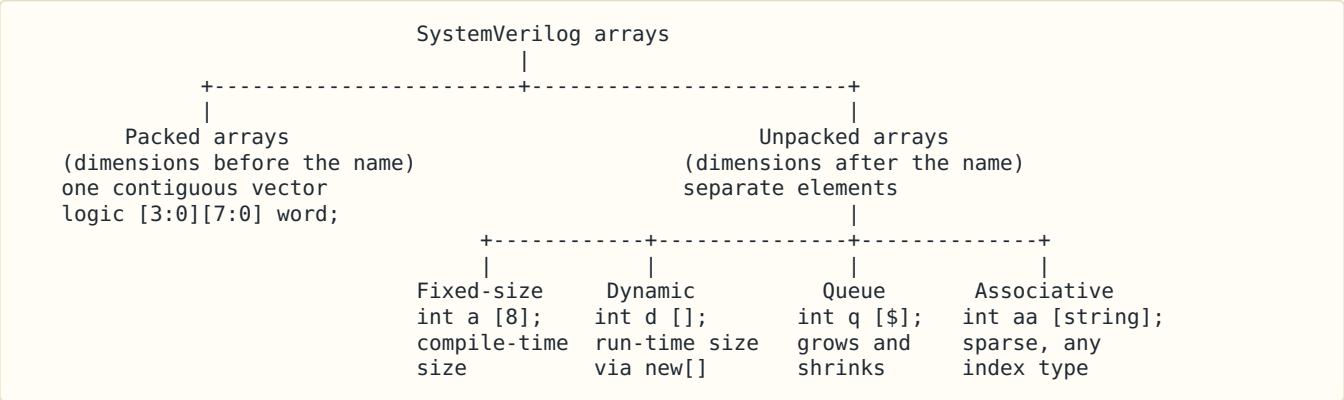
SystemVerilog has one array kind for each of these jobs.

Technical definition

Kind	Declaration	Size	Index	Storage allocated
Fixed-size (static) array	`int a [8];`	Fixed at compile time	Integer, contiguous	At elaboration, for all elements
Dynamic array	`int d [];`	Set and changed at run time with `new[]`	Integer, contiguous, from 0	When `new[]` is called
Queue	`int q [$];`	Grows and shrinks automatically	Integer, contiguous, from 0	Automatically, as elements are added
Associative array	`int aa [string];`	Grows as entries are written	Any type (integer, string, class handle...), sparse	Per entry, when the entry is first written

All four are **unpacked** arrays: the bracket that makes them an array comes after the name. The element type can be anything, including a packed vector (`logic [7:0] mem [256]`), a struct, a class handle or another array.

Packed arrays (dimensions before the name, as in `logic [3:0][7:0] word`) are a separate concept: one contiguous vector with sub-fields. They were covered in Module 1 and are summarised again in Section 1.3.



Why so many kinds are needed

Each kind trades flexibility against speed and memory.

Requirement	Best kind	Reason
Size known when the code is written	Fixed-size	Fastest, simplest, and the only unpacked kind that is synthesizable
Size known only at run time, then mostly constant	Dynamic array	One contiguous block; fast indexed access; resized explicitly
Elements added and removed all the time, mainly at the ends	Queue	Insertion and removal at either end without reallocating
Huge or non-numeric index space, few entries used	Associative array	Memory is used only for the entries that exist

Synthesizable or simulation-only

Kind	Synthesizable
Packed array	Yes
Fixed-size unpacked array	Yes
Dynamic array	No
Queue	No
Associative array	No

Dynamic arrays, queues and associative arrays exist only in testbench code.

Interview questions

Q1. What types of arrays does SystemVerilog have? Packed arrays and unpacked arrays. Unpacked arrays are of four kinds: fixed-size (static) arrays, dynamic arrays, queues and associative arrays.

Q2. Which arrays are synthesizable? Packed arrays and fixed-size unpacked arrays. Dynamic arrays, queues and associative arrays are simulation-only because their size is not known at compile time.

Q3. In one sentence each, when would you use a dynamic array, a queue and an associative array? A dynamic array when the size is decided at run time but then stays mostly constant, such as a random-length payload. A queue when elements are continuously added and removed, such as the expected-result list of a

scoreboard. An associative array when the index space is large and sparsely used or is not an integer, such as a memory model or a lookup keyed by a string.

1.2 Static (Fixed-Size) Arrays

In plain English

A static array has a size that is written in the source code and never changes. Declaring `int a [5];` creates exactly five `int` elements, numbered 0 to 4, for the whole simulation.

"Static" here means "fixed size". It is not related to the `static` keyword used for variable lifetime (Module 3) or class members (Module 4).

Technical definition

A fixed-size array is an unpacked array whose dimension ranges are constant expressions. Its storage is allocated before simulation starts and exists for the lifetime of the variable. Each element has the default value of the element type until it is assigned.

Syntax

```
int         a [0:4];              // 5 elements, indices 0 to 4
int         b [5];                // C-style shorthand, identical to [0:4]
logic [7:0] mem [0:255];          // 256 elements of 8 bits (a memory)
int         c [2:6];              // 5 elements, indices 2 to 6
int         d [6:2];              // 5 elements, indices 6 down to 2
int         e [5] = '{1, 2, 3, 4, 5};        // initialised in the declaration
parameter int N = 16;
bit [3:0]   f [N];                // the size may be a parameter (a constant)
```

The size must be a constant: a literal, a `parameter`, a `localparam`, or an expression of constants. A variable is not allowed; that is what dynamic arrays are for.

How it works internally

```
int a [5] = '{10, 20, 30, 40, 50};

index :     0        1        2        3        4
          +--------+--------+--------+--------+--------+
a         |   10   |   20   |   30   |   40   |   50   |     5 x 32 bits, allocated
          +--------+--------+--------+--------+--------+     once, before time 0

a[3]  ->  the simulator computes (start of a) + 3 elements : direct access, no search
```

- The memory is one block whose size never changes, so access by index is the fastest of all array kinds.
- The simulator knows the bounds at compile time. An out-of-range **constant** index is usually reported at compile time; an out-of-range **variable** index is detected at run time (read returns the default value X or 0, write is ignored, normally with a warning).

Operations on a whole fixed-size array

Operation	Example	Notes
Element read / write	`a[2] = 99;`	
Slice (a contiguous part)	`a[1:3]`	An array of 3 elements
Whole-array assignment	`a = b;`	Both must have the same element type and the same number of elements
Assignment pattern	`a = '{1, 2, 3, 4, 5};`	One value per element
Default pattern	`a = '{default: 0};`	Every element
Comparison	`a == b`, `a != b`	Element by element; result is 1 bit
Number of elements	`$size(a)`	The `size()` method is **not** defined for fixed-size arrays
Arithmetic on the whole	`a + 1`	**Not allowed**: an unpacked array is not a number
Array methods	`a.sum()`, `a.sort()`, `a.find with (...)`	Part 4 of this module

Complete example

tb_static_array.sv

```
module tb_static_array;

  int         a [5];                          // indices 0 to 4
  int         b [5] = '{10, 20, 30, 40, 50};
  logic [7:0] mem [0:3];
  int         idx;

  initial begin
    $display("1. defaults: a[0]=%0d  mem[0]=%h", a[0], mem[0]);

    a    = '{default: 7};
    a[2] = 99;
    $display("2. a = %p", a);

    a    = b;                 // whole-array copy
    a[0] = -1;
    $display("3. a[0]=%0d  b[0]=%0d", a[0], b[0]);

    $display("4. a == b : %b    a[1:3] == b[1:3] : %b", a == b, a[1:3] == b[1:3]);
    $display("5. $size(a) = %0d", $size(a));

    foreach (mem[i]) mem[i] = 8'h10 + i;
    $display("6. mem[3]=%h  mem[1][0]=%b", mem[3], mem[1][0]);

    idx = 9;
    $display("7. out of range: a[9]=%0d  mem[9]=%h", a[idx], mem[idx]);
  end

endmodule
```

EXPECTED OUTPUT

```
1. defaults: a[0]=0  mem[0]=xx
2. a = '{7, 7, 99, 7, 7}
3. a[0]=-1  b[0]=10
4. a == b : 0    a[1:3] == b[1:3] : 1
5. $size(a) = 5
6. mem[3]=13  mem[1][0]=1
7. out of range: a[9]=0  mem[9]=xx
```

Line by line

Line	Explanation
1	Elements take the default of their type: `int` is two-state (0); `logic [7:0]` is four-state (X).
2	`'{default: 7}` sets all five elements. Then one element is overwritten. `%p` prints the whole array.
3	`a = b` copies all five values. The arrays stay independent: changing `a[0]` does not change `b[0]`.
4	The arrays now differ in element 0, so `a == b` is 0. The slices `[1:3]` hold 20, 30, 40 in both, so they are equal.
5	`$size` returns the number of elements.
6	`foreach` visits indices 0 to 3 and stores `10`, `11`, `12`, `13` (hex). `mem[1]` is `11` hex = `0001_0001`; its bit 0 is 1. The unpacked index comes first, then the packed bit-select.
7	Index 9 does not exist. Reading returns the default value of the element type: 0 for `int`, X for `logic`. Simulators print a warning and continue. A variable index is used here because many simulators reject an out-of-range constant index at compile time.

Common mistake: Calling `a.size()` on a fixed-size array. The `size()` method belongs to dynamic arrays, queues and associative arrays. For a fixed-size array use `$size(a)`. Some simulators accept `a.size()` as an extension, but the standard does not define it, so portable code uses `$size`.

Common mistake: `int a [5]; a = 0;`. An unpacked array cannot be assigned a single number. Use `a = '{default: 0};`.

Verification application: Fixed-size arrays are used where the size is a property of the design: a model of a 32-entry register file (`logic [31:0] regs [32]`), a small lookup table of expected values, an array of virtual interfaces for a DUT with a fixed number of ports (`virtual alu_if vif [4]`), or an array of per-port counters.

Interview questions

Q1. What is a static array? An unpacked array whose size is a compile-time constant. Its storage is allocated before simulation starts and cannot be resized.

Q2. What is the difference between `int a [5];` and `int a [0:4];`? None. `[5]` is shorthand for `[0:4]`.

Q3. How do you get the number of elements of a fixed-size array? `$size(a)`. The `size()` method is not defined for fixed-size arrays.

Q4. Can the size of a fixed-size array be a variable? No. It must be a constant expression such as a literal or a parameter. Use a dynamic array when the size is known only at run time.

Q5. What happens when you read beyond the end of a fixed-size array? The read returns the default value of the element type (X for four-state, 0 for two-state) and a write is ignored. The simulation continues, usually with a warning.

1.3 Packed Arrays and Unpacked Arrays Revisited

Module 1, Sections 3.1 and 3.2, explained the difference in detail. This section restates what matters for the rest of this module and adds the points that concern array methods.

The two families

Property	Packed array	Unpacked array
Declaration	`logic [3:0][7:0] p;`	`logic [7:0] u [4];`
What it is	One contiguous vector, viewed as fields	A collection of separate elements
Element types allowed	Bit-level types only	Any type
Is a number (integral)	Yes: `p + 1`, `p == 32'h0`, `p = 32'hDEADBEEF`	No
Size	Always fixed	Fixed, or dynamic / queue / associative
Printing	`%h`, `%b`, `%d`	`%p` or a loop
Array methods (`sum`, `sort`, `find`...)	**Not available**	Available
`foreach`	Allowed (iterates over the bits or fields)	Allowed
Query functions (`$size`, `$left`...)	Work	Work
Synthesizable	Yes	Fixed-size only

> **Key point:** The built-in array methods of Part 4 apply to unpacked arrays only. `logic [7:0] v; v.sum()` is illegal. To count the ones in a vector use `$countones(v)`; to reduce a vector use the reduction operators of Module 1 (`^v`, `&v`, `|v`).

Combining both

Most real declarations have a packed part (the width of one element) and an unpacked part (how many elements):

```
logic [7:0] mem [0:255];         // 256 unpacked elements, each a packed byte
bit [31:0]  regs [16];           // 16 registers of 32 bits
bit [7:0]   payload [];          // dynamic array of bytes
logic [8:0] exp_q [$];           // queue of 9-bit expected results
bit [7:0]   sparse_mem [int];    // associative array of bytes
```

The index order rule from Module 1 applies to every kind: **unpacked indices first, then packed indices.** `mem[17][3]` is bit 3 of element 17.

tb_packed_vs_unpacked.sv

```
module tb_packed_vs_unpacked;

  logic [3:0][7:0] p;           // packed: one 32-bit vector made of 4 bytes
  logic [7:0]      u [4];       // unpacked: 4 separate bytes, indices 0 to 3

  initial begin
    p = 32'hA1B2_C3D4;                      // assign as a number
    u = '{8'hA1, 8'hB2, 8'hC3, 8'hD4};      // assign per element

    $display("1. p[3]=%h p[0]=%h   u[0]=%h u[3]=%h", p[3], p[0], u[0], u[3]);
    $display("2. p + 1 = %h", p + 1);       // legal only for the packed array

    foreach (p[i]) $write("p[%0d]=%h ", i, p[i]);
    $display("");
    foreach (u[i]) $write("u[%0d]=%h ", i, u[i]);
    $display("");

    $display("3. $bits(p)=%0d  $bits(u)=%0d  $size(p)=%0d  $size(u)=%0d",
             $bits(p), $bits(u), $size(p), $size(u));
  end

endmodule
```

EXPECTED OUTPUT

```
1. p[3]=a1 p[0]=d4   u[0]=a1 u[3]=d4
2. p + 1 = a1b2c3d5
p[3]=a1 p[2]=b2 p[1]=c3 p[0]=d4
u[0]=a1 u[1]=b2 u[2]=c3 u[3]=d4
3. $bits(p)=32  $bits(u)=32  $size(p)=4  $size(u)=4
```

Explanation

Line	Explanation
1	In the packed array the dimension is `[3:0]`, so the leftmost byte of the number (`A1`) is `p[3]`. In the unpacked array the dimension is `[0:3]`, so the first value of the pattern (`A1`) is `u[0]`. The same data, opposite numbering, purely because of how each range was declared.
2	Arithmetic on the whole value is legal for `p`. `u + 1` would not compile.
`foreach` lines	`foreach` walks from the left bound of the declared range to the right bound: 3, 2, 1, 0 for `p`; 0, 1, 2, 3 for `u`. `$write` prints without a newline; the empty `$display` ends the line.
3	Both hold 32 bits and both have 4 elements in their first dimension.

Interview questions

Q1. Can you call `sum()` or `sort()` on a packed array? No. Array methods are defined for unpacked arrays only. A packed array is a single integral value.

Q2. In what order does `foreach` visit `logic [7:0] v`? From the left bound to the right bound of the declaration: bit 7 first, bit 0 last.

Q3. `bit [7:0] a [4] = '{1, 2, 3, 4};` and `bit [3:0][7:0] b = {8'd1, 8'd2, 8'd3, 8'd4};`. Which element holds 1 in each? `a[0]` holds 1 because the assignment pattern fills the range `[0:3]` left to right. `b[3]` holds 1 because the first item of a concatenation is the most significant and the packed range is `[3:0]`.

1.4 Multidimensional Arrays

In plain English

A one-dimensional array is a row of elements. A two-dimensional array is a grid: rows and columns. A three-dimensional array is a stack of grids. In SystemVerilog a multidimensional array is simply **an array whose elements are themselves arrays**.

Technical definition

A multidimensional unpacked array is declared by writing more than one dimension after the name. Each dimension can independently be fixed-size, dynamic, a queue or associative. The dimensions are indexed left to right as written.

Syntax

```
int       m [2][3];              // 2 rows, 3 columns: 6 ints
int       c [4][8][2];           // 3 dimensions: 64 ints
bit [7:0] frame [10][64];        // 10 rows of 64 bytes
int       jag [][];              // dynamic array of dynamic arrays (jagged rows)
int       per_port_q [4][$];     // 4 queues
int       hist [string][int];    // associative array of associative arrays
```

How it works internally

```
int m [2][3] = '{ '{1, 2, 3},
                  '{4, 5, 6} };

               column:    0     1     2
                        +-----+-----+-----+
         m[0] (row 0) |  1  |  2  |  3  |        m[0]    is an array of 3 ints
                        +-----+-----+-----+
         m[1] (row 1) |  4  |  5  |  6  |        m[1][2] is the int 6
                        +-----+-----+-----+

The FIRST index selects the row (the outer array).
The SECOND index selects the element inside that row.
$size(m, 1) = 2 (rows)        $size(m, 2) = 3 (columns)
```

For fixed-size arrays the rows all have the same length. When an inner dimension is dynamic (`int jag [][]`), each row is a separate dynamic array with its own size, so the rows can have different lengths. Such an array is called **jagged**.

```
int jag [][];
jag = new[3];             // 3 rows, each still empty
jag[0] = new[1];
jag[1] = new[2];
jag[2] = new[3];

     jag[0] -> [ . ]
     jag[1] -> [ . | . ]
     jag[2] -> [ . | . | . ]
```

Initialisation with nested assignment patterns

One level of `'{ }` is needed per dimension:

```
int m [2][3] = '{ '{1, 2, 3}, '{4, 5, 6} };      // row by row
int z [2][3] = '{default: 0};                     // every element
int r [2][3] = '{ '{default: 1}, '{7, 8, 9} };   // row 0 all ones, row 1 listed
int s [2][3] = '{2{ '{3{5}} }};                   // replication: every element is 5
```

Accessing parts of a multidimensional array

Expression	Meaning for `int m [2][3]`
`m`	The whole array, 2 × 3
`m[1]`	Row 1: an array of 3 ints. Can be copied to or from an `int x [3]`
`m[1][2]`	One element
`m[0][0:1]`	A slice of row 0: 2 elements
`foreach (m[i, j])`	Visits every element; `i` is the row, `j` the column
`foreach (m[i])`	Visits every row

Complete example

tb_multidim.sv

```
module tb_multidim;

  int m [2][3] = '{ '{1, 2, 3}, '{4, 5, 6} };
  int row [3];
  int jag [][];
  int total;

  initial begin
    $display("1. m[1][2] = %0d", m[1][2]);

    row = m[0];                                // copy one whole row
    $display("2. row = %p", row);

    foreach (m[i, j]) m[i][j] = m[i][j] * 10;
    foreach (m[i]) $display("3. row %0d : %0d %0d %0d", i, m[i][0], m[i][1], m[i][2]);

    $display("4. rows = %0d, columns = %0d", $size(m, 1), $size(m, 2));

    total = 0;
    for (int i = 0; i < $size(m, 1); i++)
      for (int j = 0; j < $size(m, 2); j++)
        total += m[i][j];
    $display("5. total = %0d", total);

    m    = '{default: 0};
    m[1] = '{7, 8, 9};
    $display("6. m = %p", m);

    // jagged array: every row has a different length
    jag = new[3];
    foreach (jag[i]) begin
      jag[i] = new[i + 1];
      for (int j = 0; j < jag[i].size(); j++) jag[i][j] = i * 10 + j;
    end
    foreach (jag[i]) $display("7. jag[%0d] has %0d elements: %p", i, jag[i].size(), jag[i]);
  end

endmodule
```

EXPECTED OUTPUT

```
1. m[1][2] = 6
2. row = '{1, 2, 3}
3. row 0 : 10 20 30
3. row 1 : 40 50 60
4. rows = 2, columns = 3
5. total = 210
6. m = '{'{0, 0, 0}, '{7, 8, 9}}
7. jag[0] has 1 elements: '{0}
7. jag[1] has 2 elements: '{10, 11}
7. jag[2] has 3 elements: '{20, 21, 22}
```

Line by line

Line	Explanation
1	Row 1, column 2.
2	`m[0]` is itself an array of three ints, so it can be assigned to `row`.
3	`foreach (m[i, j])` is the multidimensional form: the loop variables are listed inside **one** pair of brackets, separated by a comma. It visits (0,0), (0,1), (0,2), (1,0), (1,1), (1,2). Each element is multiplied by 10.
4	The second argument of `$size` selects the dimension.
5	The same traversal with nested `for` loops. 10 + 20 + 30 + 40 + 50 + 60 = 210.
6	`'{default: 0}` reaches every element of every dimension. Then row 1 is replaced as a whole.
7	`jag = new[3]` creates three empty rows. Each row is then allocated with its own size, and an ordinary `for` loop bounded by `jag[i].size()` fills that row.

Interview trap: The loop is written `foreach (m[i, j])`, not `foreach (m[i][j])`. The standard defines only the first form: all loop variables inside one pair of brackets, separated by commas. Some simulators accept the second form as an extension, but it is not portable.

Common mistake: Forgetting that an inner dynamic dimension must be allocated separately. After `jag = new[3];` the rows exist but each has size 0; writing `jag[0][0] = 5;` is an out-of-range write and is ignored.

Verification application: Two-dimensional arrays model anything with rows and columns: a pixel frame in a video testbench (`bit [7:0] frame [HEIGHT][WIDTH]`), a set of per-channel payloads, or a memory organised in pages. A fixed array of queues, `alu_txn exp_q [4][$]`, is the usual scoreboard structure for a design with four output ports: one expected queue per port.

Interview questions

Q1. How do you declare and initialise a 2 × 3 array? `int m [2][3] = '{ '{1, 2, 3}, '{4, 5, 6} };`

Q2. How do you iterate over a two-dimensional array with `foreach`? `foreach (m[i, j])`, with both loop variables inside one pair of brackets.

Q3. What is the difference between `int a [2][3];` and `bit [1:0][2:0] a;`? The first is an unpacked two-dimensional array of six separate 32-bit ints. The second is a packed two-dimensional array: one 6-bit vector divided into two 3-bit fields.

Q4. What is a jagged array and how do you create one? An array of arrays whose rows have different lengths. Declare `int j [][];`, allocate the outer dimension with `j = new[n];`, then allocate each row separately with `j[i] = new[len];`.

Q5. How do you find the number of columns of `int m [4][7]`? `$size(m, 2)`, which returns 7.

1.5 Array Query Functions: $size, $dimensions, $left, $right, $low, $high

In plain English

These system functions answer questions about the **shape** of an array: how many dimensions it has, how many elements a dimension has, and what its index bounds are. They let you write loops and checks that keep working when a declaration is changed.

Technical definition

Function	Returns
`$dimensions(arr)`	Total number of dimensions, packed and unpacked together
`$unpacked_dimensions(arr)`	Number of unpacked dimensions only
`$size(arr, dim)`	Number of elements in dimension `dim`
`$left(arr, dim)`	The left bound of the dimension, as written in the declaration
`$right(arr, dim)`	The right bound of the dimension, as written in the declaration
`$low(arr, dim)`	The smaller of the two bounds
`$high(arr, dim)`	The larger of the two bounds
`$increment(arr, dim)`	1 if `$left` >= `$right`, otherwise -1
`$bits(arr)`	Total number of bits in the whole array

The `dim` argument is optional and defaults to 1.

How dimensions are numbered: dimension 1 is the **leftmost unpacked** dimension. Numbering continues through the remaining unpacked dimensions left to right, then through the packed dimensions left to right.

```
logic [3:0][7:0]  arr  [2][5];
       |     |          |  |
   dim 3   dim 4     dim 1  dim 2

$dimensions(arr)           = 4
$unpacked_dimensions(arr)  = 2
$size(arr, 1) = 2     $size(arr, 2) = 5     $size(arr, 3) = 4     $size(arr, 4) = 8
```

This is the same order in which you write the indices when accessing an element: `arr[i][j][k][l]`.

$left / $right versus $low / $high

`$left` and `$right` follow the declaration. `$low` and `$high` follow the numeric values.

Declaration	`$left`	`$right`	`$low`	`$high`	`$increment`	`$size`
`logic [7:0] v`	7	0	0	7	1	8
`logic [0:7] v`	0	7	0	7	-1	8
`int a [0:3]` or `int a [4]`	0	3	0	3	-1	4
`int a [6:2]`	6	2	2	6	1	5
Dynamic array or queue with N elements	0	N-1	0	N-1	-1	N

Remember `$increment` like this: stepping from `$left` to `$right` means **subtracting** `$increment`. For `[7:0]` the increment is 1, and 7 - 1 = 6 moves toward 0.

$size() versus size()

	`$size(arr)`	`arr.size()`
What it is	A system function	A built-in method
Works on	Every array: packed, fixed-size, dynamic, queue, associative	Dynamic arrays, queues and associative arrays only
Can select a dimension	Yes: `$size(arr, 2)`	No: it reports the dimension it is called on (`arr[i].size()` for an inner one)
Typical use	Fixed-size arrays, generic code, multidimensional arrays	Dynamic arrays and queues

For an associative array, `num()` and `size()` both return the number of entries.

Complete example

tb_array_query.sv

```
module tb_array_query;

  logic [7:0]      mem  [0:3];        // dim 1 = [0:3] (unpacked), dim 2 = [7:0] (packed)
  logic [3:0][7:0] word;              // dim 1 = [3:0], dim 2 = [7:0] (both packed)
  bit [15:0]       grid [2][6:4];     // dim 1 = [0:1], dim 2 = [6:4], dim 3 = [15:0]
  int              dyn  [];

  initial begin
    $display("--- mem : logic [7:0] mem [0:3] ---");
    $display("dimensions=%0d unpacked=%0d bits=%0d",
             $dimensions(mem), $unpacked_dimensions(mem), $bits(mem));
    $display("dim 1: size=%0d left=%0d right=%0d low=%0d high=%0d increment=%0d",
             $size(mem), $left(mem), $right(mem), $low(mem), $high(mem), $increment(mem));
    $display("dim 2: size=%0d left=%0d right=%0d low=%0d high=%0d increment=%0d",
             $size(mem, 2), $left(mem, 2), $right(mem, 2), $low(mem, 2), $high(mem, 2),
             $increment(mem, 2));

    $display("--- word : logic [3:0][7:0] word ---");
    $display("dimensions=%0d unpacked=%0d size1=%0d size2=%0d left1=%0d",
             $dimensions(word), $unpacked_dimensions(word), $size(word, 1), $size(word, 2),
             $left(word, 1));

    $display("--- grid : bit [15:0] grid [2][6:4] ---");
    $display("dimensions=%0d size1=%0d size2=%0d size3=%0d",
             $dimensions(grid), $size(grid, 1), $size(grid, 2), $size(grid, 3));
    $display("dim 2: left=%0d right=%0d low=%0d high=%0d increment=%0d",
             $left(grid, 2), $right(grid, 2), $low(grid, 2), $high(grid, 2),
             $increment(grid, 2));

    $display("--- dyn : int dyn [] ---");
    dyn = new[5];
    $display("size=%0d $size=%0d left=%0d right=%0d high=%0d",
             dyn.size(), $size(dyn), $left(dyn), $right(dyn), $high(dyn));
  end

endmodule
```

EXPECTED OUTPUT

```
--- mem : logic [7:0] mem [0:3] ---
dimensions=2 unpacked=1 bits=32
dim 1: size=4 left=0 right=3 low=0 high=3 increment=-1
dim 2: size=8 left=7 right=0 low=0 high=7 increment=1
--- word : logic [3:0][7:0] word ---
dimensions=2 unpacked=0 size1=4 size2=8 left1=3
--- grid : bit [15:0] grid [2][6:4] ---
dimensions=3 size1=2 size2=3 size3=16
dim 2: left=6 right=4 low=4 high=6 increment=1
--- dyn : int dyn [] ---
size=5 $size=5 left=0 right=4 high=4
```

Explanation

Output line	Explanation
`mem`, first line	One unpacked dimension plus one packed dimension gives 2. Total bits 4 × 8 = 32.
`mem` dim 1	The unpacked range `[0:3]`: left 0, right 3. Left is smaller than right, so increment is -1.
`mem` dim 2	The packed range `[7:0]`: left 7, right 0, low 0, high 7, increment 1.
`word`	Both dimensions are packed, so the unpacked count is 0. Dimension 1 is `[3:0]`.

Output line	Explanation
`grid`	Three dimensions: `[2]` means `[0:1]`, then `[6:4]`, then the packed `[15:0]`. Dimension 2 has left 6, right 4.
`dyn`	A dynamic array is always indexed from 0 to size-1. `$size` and the `size()` method agree.

Writing loops that do not depend on the declaration

```
// Works for [0:N-1] and for [N-1:0], and for any bounds
for (int i = $low(arr); i <= $high(arr); i++) ...

// Visit in declaration order (the order foreach uses)
for (int i = $left(arr); i != $right(arr) - $increment(arr); i -= $increment(arr)) ...
```

In practice `foreach (arr[i])` does the second form for you and is what you should write. The query functions are needed when you must loop over only part of an array, loop backwards, or check a size.

Interview trap: "What does `$dimensions` return for `int a [4];`?" The answer is 2, not 1. `int` is itself a 32-bit packed vector, so there is one unpacked dimension and one packed dimension. `$unpacked_dimensions(a)` is 1. The same reasoning gives `$dimensions(bit [7:0] b [4])` = 2 and `$dimensions(bit c [4])` = 1.

Common mistake: Writing loops with hard-coded bounds, `for (int i = 0; i < 16; i++)`. When someone changes the array to 32 entries, the loop silently covers half of it. Use `foreach`, `$size(arr)` or `arr.size()`.

Verification application: Generic checkers and utilities use the query functions so that they work for any array passed in. A common assertion in a testbench is a size check before a copy or comparison: `if ($size(actual) != $size(expected)) $error("length mismatch");`.

Interview questions

Q1. What is the difference between `$size` and `size()`? `$size` is a system function that works on any array, including packed and fixed-size arrays, and can report any dimension. `size()` is a method available only on dynamic arrays, queues and associative arrays.

Q2. What is the difference between `$left` and `$low`? `$left` returns the bound written on the left in the declaration. `$low` returns the numerically smaller bound. For `logic [7:0] v`, `$left` is 7 and `$low` is 0.

Q3. For `bit [7:0] a [2][3];`, what are `$dimensions(a)`, `$size(a)`, `$size(a, 2)`, `$size(a, 3)` and `$bits(a)`? 3, 2, 3, 8 and 48.

Q4. How are dimensions numbered? Unpacked dimensions first, left to right, starting from 1; then the packed dimensions, left to right.

Q5. What does `$increment` return? 1 if the left bound is greater than or equal to the right bound, and -1 otherwise.

Q6. What do `$left`, `$right` and `$size` return for a queue with 6 elements? 0, 5 and 6.

Part 2: Dynamic Arrays and Queues

2.1 Dynamic Arrays

In plain English

Sometimes you do not know, when writing the code, how many elements you will need. A packet may carry 1 byte or 1500 bytes, and the length is chosen at random for each packet. A fixed-size array would have to be declared with the maximum size and would waste memory and hide the real length.

A dynamic array is an array whose size is chosen **while the simulation is running**. You declare it empty, and later tell the simulator "give me N elements now". You can ask for a different size again whenever you like.

Technical definition

A dynamic array is an unpacked array declared with empty brackets `[]`. It has no storage until the `new[]` constructor allocates it at run time. Its elements are always indexed from 0 to `size()-1`. Its size can be changed at any time by calling `new[]` again, optionally preserving the existing contents.

Why it is needed

Need	Why a fixed-size array is not enough
Random-length payloads	The length is a random variable; each transaction needs a different size
Data whose amount depends on configuration or on a file	The size is known only after the test has started
Memory efficiency	Allocating the maximum for every transaction wastes memory when thousands of transactions exist
Randomization of the size itself	`rand bit [7:0] payload [];` with a constraint on `payload.size()` lets the solver choose the length (Module 5)

Syntax

```
int       d [];                 // declaration: empty, size 0
bit [7:0] payload [];           // dynamic array of bytes

d = new[10];                    // allocate 10 elements, each set to the default value
d = new[20](d);                 // resize to 20, keeping the old 10 values
d = '{1, 2, 3};                 // assignment pattern: size becomes 3
int e [] = new[4];              // allocate in the declaration
int f [] = '{5, 6, 7};          // initialise in the declaration
```

Methods and operators

Operation	Syntax	What it does
Allocate	`d = new[n];`	Creates `n` elements with default values. Any previous contents are discarded
Allocate and copy	`d = new[n](src);`	Creates `n` elements and copies as many elements of `src` as fit, starting at index 0. Extra elements get the default value

Operation	Syntax	What it does
Size	`d.size()`	Returns the current number of elements (0 if not allocated)
Delete	`d.delete();`	Frees all elements; the size becomes 0
Copy	`d2 = d1;`	`d2` is resized to match and receives a copy of every element
Compare	`d1 == d2`	1 if the sizes are equal and every element is equal

`new[]` with square brackets is the array constructor. It is different from `new()` with round brackets, which creates a class object (Module 4).

How it works internally

```
int d [];              d : (no storage)                              size = 0

d = new[4];            d : [ 0 | 0 | 0 | 0 ]                          size = 4
                           0   1   2   3

d[1] = 7; d[3] = 9;    d : [ 0 | 7 | 0 | 9 ]

d = new[6](d);         a NEW block of 6 is allocated,
                       the old 4 values are copied into it,
                       the old block is released:
                       d : [ 0 | 7 | 0 | 9 | 0 | 0 ]                  size = 6

d = new[2](d);         d : [ 0 | 7 ]                                  size = 2
                       (only the first 2 values fit)

d = new[3];            d : [ 0 | 0 | 0 ]                              size = 3
                       (no source given: old values are LOST)

d.delete();            d : (no storage)                              size = 0
```

Important consequences:

- The elements are stored as one contiguous block, so `d[i]` is as fast as for a fixed-size array.
- Resizing means allocating a new block and copying. Doing that once per added element is slow; a queue is the right tool for element-by-element growth.
- A dynamic array **never grows by itself**. Writing to an index at or beyond `size()` is an out-of-range write: it is ignored (with a warning), and the array is not extended.

Complete example

tb_dynamic_array.sv

```
module tb_dynamic_array;

  int d [];
  int e [];
  int f [4] = '{1, 2, 3, 4};

  initial begin
    $display("1. size = %0d", d.size());

    d = new[4];
    foreach (d[i]) d[i] = (i + 1) * 10;
    $display("2. d = %p  size = %0d", d, d.size());

    d = new[6](d);                          // grow, keep contents
    $display("3. d = %p", d);

    d = new[2](d);                          // shrink, keep the first two
    $display("4. d = %p", d);

    d = new[3];                             // no source: contents are lost
    $display("5. d = %p", d);

    e    = d;                               // independent copy, e is resized to 3
    e[0] = 5;
    $display("6. d[0] = %0d  e[0] = %0d", d[0], e[0]);

    d = '{7, 8, 9, 10, 11};                 // the pattern sets the size
    $display("7. size = %0d", d.size());

    d = f;                                  // fixed-size array copied into a dynamic array
    $display("8. d = %p", d);

    d.delete();
    $display("9. size = %0d", d.size());
  end

endmodule
```

EXPECTED OUTPUT

```
1. size = 0
2. d = '{10, 20, 30, 40}  size = 4
3. d = '{10, 20, 30, 40, 0, 0}
4. d = '{10, 20}
5. d = '{0, 0, 0}
6. d[0] = 0  e[0] = 5
7. size = 5
8. d = '{1, 2, 3, 4}
9. size = 0
```

Line by line

Line	Explanation
1	A declared but unallocated dynamic array has size 0.
2	`new[4]` creates four `int` elements, each 0. The `foreach` fills them with 10, 20, 30, 40.
3	`new[6](d)` allocates six elements and copies the existing four. The two new elements are 0.
4	`new[2](d)` keeps only the first two values.
5	`new[3]` without a source allocates three fresh elements. The old 10 and 20 are gone.

Line	Explanation
6	Assigning one dynamic array to another resizes the destination and copies the values. The two arrays are independent afterwards.
7	An assignment pattern with five values makes the size 5. No `new[]` is needed.
8	A fixed-size array can be assigned to a dynamic array of the same element type; the dynamic array takes its size.
9	`delete()` releases the storage.

A practical example: a variable-length payload

tb_dynamic_payload.sv

```
module tb_dynamic_payload;

  bit [7:0]    payload [];
  int unsigned len;
  bit [7:0]    checksum;

  initial begin
    len     = 6;                              // in a real test this would be random
    payload = new[len];
    foreach (payload[i]) payload[i] = 8'hA0 + i;

    checksum = 8'h00;
    foreach (payload[i]) checksum ^= payload[i];

    // append the checksum: grow by one element, keeping the data
    payload = new[payload.size() + 1](payload);
    payload[payload.size() - 1] = checksum;

    $write("frame (%0d bytes):", payload.size());
    foreach (payload[i]) $write(" %h", payload[i]);
    $display("");
  end

endmodule
```

EXPECTED OUTPUT

```
frame (7 bytes): a0 a1 a2 a3 a4 a5 01
```

The payload holds `A0` to `A5`. XORing them pairwise: `A0^A1` = `01`, `^A2` = `A3`, `^A3` = `00`, `^A4` = `A4`, `^A5` = `01`. The array is then grown by one element with `new[size+1](payload)` and the checksum is stored in the new last position.

Fixed-size array versus dynamic array

Property	Fixed-size array	Dynamic array
Declaration	`int a [8];`	`int d [];`
Size decided	At compile time	At run time, with `new[]`
Can be resized	No	Yes
Storage exists	For the whole simulation	From `new[]` until `delete()` or reallocation
Index range	Any declared range, e.g. `[2:9]`	Always `0` to `size()-1`
Number of elements	`$size(a)`	`d.size()` or `$size(d)`

Property	Fixed-size array	Dynamic array
Size can be randomized	No	Yes, by constraining `d.size()`
Synthesizable	Yes	No
Typical use	Design-defined sizes: register files, port arrays	Payloads, data read from files, anything of run-time length

Assignment between fixed-size and dynamic arrays

Assignment	Legal?	Behaviour
`dyn = fixed;`	Yes	`dyn` is resized to the size of `fixed`
`dyn1 = dyn2;`	Yes	`dyn1` is resized to match `dyn2`
`fixed = dyn;`	Compiles	**Run-time error** if `dyn.size()` differs from the size of `fixed`

In all cases the element types must be equivalent.

Interview trap: "What is the difference between `d = new[10];` and `d = new[10](d);`?" The first discards the existing contents. The second preserves them. Forgetting the `(d)` when growing an array silently erases the data.

Common mistake: Using a dynamic array before allocating it. `int d []; d[0] = 5;` compiles, but at run time index 0 does not exist: the write is ignored, and a later read returns 0. Allocate with `new[]` or an assignment pattern first.

Common mistake: Appending one element at a time with `d = new[d.size() + 1](d);` inside a loop. Every iteration allocates a new block and copies the whole array. Use a queue, or allocate the final size once.

Verification application: In a packet-based testbench the transaction class contains `rand bit [7:0] payload [];` together with `constraint len_c { payload.size() inside {[1:64]}; }`. Calling `randomize()` first chooses the size, allocates the array, and then randomizes every element. The driver walks the array with `foreach` and sends one byte per clock. Module 5 covers this in detail.

Interview questions

Q1. What is a dynamic array? An unpacked array whose size is not fixed at compile time. It is declared with empty brackets, allocated at run time with `new[]`, and can be resized or deleted during simulation.

Q2. How do you allocate, resize and free a dynamic array? `d = new[n];` allocates. `d = new[m](d);` resizes while keeping the contents. `d.delete();` frees it.

Q3. What does `new[8](d)` do if `d` has 10 elements? If `d` has 5? With 10 elements, the first 8 are copied and the last two are dropped. With 5 elements, all five are copied and the remaining three elements get the default value.

Q4. What is the default size of a dynamic array? Zero.

Q5. Can you write to `d[d.size()]` to append an element? No. A dynamic array does not grow automatically; that write is out of range and is ignored. (A queue does allow `q[$+1] = x`.)

Q6. Is a dynamic array synthesizable? No. Its size is determined at run time, which hardware cannot do.

Q7. How do you copy a dynamic array? `copy = orig;`. The destination is resized automatically and receives its own copy of the elements. `copy = new[orig.size()](orig);` does the same.

Q8. Can a dynamic array be an element of another array, or contain other arrays? Yes. `int d [][];` is a dynamic array of dynamic arrays, `int q [][$];` is a dynamic array of queues, and `int a [4][];` is a fixed array of four dynamic arrays. Each inner array is allocated separately.

2.2 Queues

In plain English

A queue is a list that can grow and shrink freely. You can add an element at the back or the front, remove one from the back or the front, and insert or delete in the middle. It never needs to be allocated: it starts empty and adjusts its own size as elements come and go.

Two everyday patterns are built from it:

- **First-in, first-out (FIFO):** add at the back, remove from the front. A line of people at a counter.
- **Last-in, first-out (LIFO, a stack):** add at the back, remove from the back. A pile of plates.

Technical definition

A queue is a variable-size, ordered collection of elements of the same type, declared with `[$]`. Elements are indexed from 0 (the first element) to `$` (the last element). The queue supports constant-time insertion and removal at both ends and offers built-in methods for these operations. Storage is managed automatically by the simulator.

Why it is needed

A testbench is full of data that arrives and leaves at unpredictable times:

- The scoreboard receives an expected result whenever a transaction is driven, and removes it when the DUT produces the corresponding output.
- A reference model of a FIFO stores written data and returns it on reads.
- A driver keeps a list of pending requests.

A dynamic array would need a reallocation and full copy for every element added or removed. A queue does this efficiently and with far less code.

Syntax

```
int       q [$];                    // unbounded queue of int, initially empty
bit [7:0] byte_q [$];               // queue of bytes
string    names [$] = {"a", "b"};   // initialised queue
int       bq [$:7];                 // bounded queue: max index 7, at most 8 elements
alu_txn   exp_q [$];                // queue of class handles (transactions)
```

The `$` symbol has two meanings: in the declaration it marks the array as a queue; in an index expression it means "the index of the last element".

Queue methods

Method	Prototype	What it does
`size()`	`function int size();`	Number of elements
`push_back(item)`	`function void push_back(T item);`	Adds `item` at the end
`push_front(item)`	`function void push_front(T item);`	Adds `item` at the beginning; existing elements move up one index
`pop_back()`	`function T pop_back();`	Removes and returns the last element
`pop_front()`	`function T pop_front();`	Removes and returns the first element
`insert(index, item)`	`function void insert(int index, T item);`	Inserts `item` so that it becomes element `index`; elements from that position onward move up
`delete(index)`	`function void delete(int index);`	Removes the element at `index`; later elements move down
`delete()`	`function void delete();`	Removes all elements

In addition a queue supports everything a dynamic array supports: indexing, slicing, `foreach`, whole-queue copy and comparison, and the array methods of Part 4 (`find`, `sort`, `sum` and so on).

Indexing, slices and concatenation

Expression	Meaning
`q[0]`	First element
`q[$]`	Last element
`q[$-1]`	Second-to-last element
`q[1:3]`	A new queue holding elements 1, 2 and 3
`q[2:$]`	From element 2 to the end
`q[0:$-1]`	Everything except the last element
`q[$+1] = x;`	Appends `x` (the only index beyond the end that may be written)
`q = {};`	Empties the queue
`q = {q, x};`	Same as `q.push_back(x)`
`q = {x, q};`	Same as `q.push_front(x)`
`q = q[1:$];`	Same as discarding `q.pop_front()`
`q = q[0:$-1];`	Same as discarding `q.pop_back()`
`q = {q[0:i-1], x, q[i:$]};`	Same as `q.insert(i, x)`
`q = {q[0:i-1], q[i+1:$]};`	Same as `q.delete(i)`

Queues are written with **plain braces** `{ }` (an unpacked array concatenation), not with `'{ }`, because items and whole queues can be mixed freely inside the braces: `q = {q1, 5, q2};`. An assignment pattern `'{1, 2, 3}` is also accepted for a list of plain elements.

A slice whose bounds are in the wrong order or completely outside the queue (`q[3:1]`, or `q[5:9]` on a 3-element queue) yields an empty queue.

How it works internally

```
int q [$];                      q : (empty)                          size 0

q.push_back(10);                q : [10]
q.push_back(20);                q : [10 | 20]
q.push_back(30);                q : [10 | 20 | 30]
                                     0    1    2  <- index      q[$] is 30

q.push_front(5);                q : [ 5 | 10 | 20 | 30]          every index shifts by 1
                                     0    1    2    3

x = q.pop_front();   x = 5      q : [10 | 20 | 30]
x = q.pop_back();    x = 30     q : [10 | 20]

q.insert(1, 15);                q : [10 | 15 | 20]               15 becomes element 1
q.delete(0);                    q : [15 | 20]
```

The standard does not prescribe an implementation, only the behaviour: adding and removing at either end takes constant time, and access by index is efficient. Simulators typically use a double-ended buffer. Inserting or deleting **in the middle** requires moving the elements after that point, so it costs time proportional to the size.

```
      front                                      back
      push_front --> +----+----+----+----+----+ <-- push_back
      pop_front  <-- | q0 | q1 | q2 | q3 | q4 | --> pop_back
                     +----+----+----+----+----+
                       0    1    2    3    $

      FIFO  : push_back  + pop_front      (or push_front + pop_back)
      Stack : push_back  + pop_back       (or push_front + pop_front)
```

Complete example

tb_queue.sv

```
module tb_queue;

  int q [$];
  int x;

  initial begin
    q = {10, 20, 30};
    $display("1.  q = %p  size = %0d", q, q.size());

    q.push_back(40);          $display("2.  push_back(40)   : %p", q);
    q.push_front(5);          $display("3.  push_front(5)   : %p", q);

    x = q.pop_front();        $display("4.  pop_front -> %0d : %p", x, q);
    x = q.pop_back();         $display("5.  pop_back  -> %0d : %p", x, q);

    q.insert(1, 15);          $display("6.  insert(1, 15)   : %p", q);
    q.delete(2);              $display("7.  delete(2)       : %p", q);

    $display("8.  first = %0d  last = %0d  second-to-last = %0d", q[0], q[$], q[$-1]);

    q = {q, 50, 60};          $display("9.  {q, 50, 60}     : %p", q);
    q = q[1:3];               $display("10. q[1:3]          : %p", q);
    q = {q[0:0], 99, q[1:$]}; $display("11. insert by slice : %p", q);
    q[$+1] = 7;               $display("12. q[$+1] = 7      : %p", q);

    q.delete();               $display("13. delete()        : size = %0d", q.size());
    x = q.pop_front();        $display("14. pop from empty  : %0d", x);
  end

endmodule
```

EXPECTED OUTPUT

```
1.  q = '{10, 20, 30}  size = 3
2.  push_back(40)   : '{10, 20, 30, 40}
3.  push_front(5)   : '{5, 10, 20, 30, 40}
4.  pop_front -> 5 : '{10, 20, 30, 40}
5.  pop_back  -> 40 : '{10, 20, 30}
6.  insert(1, 15)   : '{10, 15, 20, 30}
7.  delete(2)       : '{10, 15, 30}
8.  first = 10  last = 30  second-to-last = 15
9.  {q, 50, 60}     : '{10, 15, 30, 50, 60}
10. q[1:3]          : '{15, 30, 50}
11. insert by slice : '{15, 99, 30, 50}
12. q[$+1] = 7      : '{15, 99, 30, 50, 7}
13. delete()        : size = 0
14. pop from empty  : 0
```

Line by line

Line	Explanation
1	A queue is initialised with braces. Size 3.
2, 3	`push_back` adds at the end, `push_front` at the beginning.
4, 5	`pop_front` and `pop_back` remove an element **and return it**.
6	`insert(1, 15)` places 15 at index 1; 20 and 30 move to indices 2 and 3.
7	`delete(2)` removes the element at index 2 (the value 20).
8	`$` is the last index, `$-1` the one before it.
9	Concatenation of the queue with two more values appends them.

Line	Explanation
10	The slice `[1:3]` of `{10, 15, 30, 50, 60}` is `{15, 30, 50}`. Assigning it back shortens the queue.
11	Building a new queue from two slices and a value: `{15}`, `99`, `{30, 50}`.
12	Writing to index `$+1` appends one element.
13	`delete()` without an argument empties the queue.
14	Popping from an empty queue is not a fatal error: it returns the default value of the element type (0 for `int`) and the simulator issues a warning. A testbench must check `size()` first, because the returned 0 looks like real data.

A practical example: the expected-result queue of a scoreboard

This is the most common use of a queue in verification. When the testbench drives a transaction it computes the expected result and pushes it. When the DUT produces an output, the scoreboard pops the oldest expected value and compares. The two small functions used here are explained in Module 3; read them as named blocks of code.

tb_queue_scoreboard.sv

```
module tb_queue_scoreboard;

  logic [8:0] exp_q [$];
  int         pass_cnt, fail_cnt;

  function automatic void add_expected(logic [8:0] value);
    exp_q.push_back(value);
  endfunction

  function automatic void check_actual(logic [8:0] actual);
    logic [8:0] expected;
    if (exp_q.size() == 0) begin
      $display("FAIL: unexpected output %0d (nothing was expected)", actual);
      fail_cnt++;
      return;
    end
    expected = exp_q.pop_front();
    if (actual !== expected) begin
      $display("FAIL: actual=%0d expected=%0d", actual, expected);
      fail_cnt++;
    end
    else begin
      $display("PASS: %0d", actual);
      pass_cnt++;
    end
  endfunction

  initial begin
    add_expected(9'd300);
    add_expected(9'd17);
    add_expected(9'd255);
    $display("pending = %0d", exp_q.size());

    check_actual(9'd300);
    check_actual(9'd18);
    check_actual(9'd255);
    check_actual(9'd1);

    $display("pass=%0d fail=%0d leftover=%0d", pass_cnt, fail_cnt, exp_q.size());
  end

endmodule
```

EXPECTED OUTPUT

```
pending = 3
PASS: 300
FAIL: actual=18 expected=17
PASS: 255
FAIL: unexpected output 1 (nothing was expected)
pass=2 fail=2 leftover=0
```

Explanation

Code	Explanation
`logic [8:0] exp_q [$];`	A queue of 9-bit expected ALU results.
`exp_q.push_back(value);`	New expectations go to the back.
`if (exp_q.size() == 0)`	The DUT produced an output that nothing predicted. Without this check, `pop_front()` would return 0 and the comparison would be meaningless.
`expected = exp_q.pop_front();`	The oldest expectation comes off the front, so results are checked in the order they were predicted (FIFO order).
`actual !== expected`	Case inequality, so that an X on the DUT output is a mismatch (Module 1, Section 4.5).
`leftover=%0d`	At the end of a test the queue must be empty. Remaining entries mean the DUT dropped outputs. This end-of-test check is a standard part of every scoreboard.

Bounded queues

```
int bq [$:2];            // highest index is 2, so the queue holds at most 3 elements

initial begin
  bq = {1, 2, 3};
  bq.push_back(4);       // the queue is full: 4 is discarded and a warning is issued
  $display("%0d", bq.size());     // 3
end
```

A bounded queue never holds more than its limit. An operation that would exceed the limit loses the elements beyond the bound and produces a warning. Bounded queues are rarely used; a testbench usually checks `size()` itself so that overflow is reported as a proper error.

Dynamic array versus queue

Property	Dynamic array	Queue
Declaration	`int d [];`	`int q [$];`
Allocation	Explicit, with `new[n]`	Automatic
Growing by one element	Reallocate and copy everything: `d = new[n+1](d)`	`push_back()`, constant time
Removing an element	Rebuild the array	`pop_front()`, `pop_back()`, `delete(i)`
Add/remove at the front	Not supported directly	`push_front()`, `pop_front()`
Access by index	Yes	Yes
Last element	`d[d.size()-1]`	`q[$]`
Literal	`'{1, 2, 3}`	`{1, 2, 3}`

Property	Dynamic array	Queue
Size can be randomized	Yes (`d.size()` in a constraint)	Yes (`q.size()` in a constraint)
Array methods (`find`, `sort`, `sum` ...)	Yes	Yes
Best for	A block of data whose size is set once	Lists that change continuously; FIFOs; stacks

Interview trap: "What does `pop_front()` return on an empty queue?" It does not stop the simulation. It returns the default value of the element type and gives a warning. For a queue of class handles the returned value is `null`, and the next statement that uses it causes a null-handle error far away from the real cause.

Common mistake: Confusing `q.delete(i)` with `q.delete()`. With an argument it removes one element. Without an argument it removes everything. A missing argument empties the whole scoreboard queue.

Common mistake: Calling `q.pop_front();` as a statement to discard an element. A function that returns a value cannot be called as a statement without a warning (or an error, depending on the tool). Write `void'(q.pop_front());`.

Common mistake: Using `insert()` and `delete(index)` heavily on a very large queue. They move all later elements. If the data is looked up by a key instead of by position, an associative array is the right structure.

Verification application: Queues appear throughout a class-based testbench: the scoreboard's expected-transaction queue (`alu_txn exp_q [$]`), the reference model of a FIFO DUT (`push_back` on write, `pop_front` on read, `size()` for the `full` and `empty` flags), a list of transactions collected by a monitor for end-of-test reporting, and the results of the array search methods, which are always returned as queues (Part 4).

Interview questions

Q1. What is a queue? A variable-size ordered collection declared with `[$]`. It grows and shrinks automatically and supports efficient insertion and removal at both ends, plus access by index.

Q2. What are the queue methods? `size()`, `push_back()`, `push_front()`, `pop_back()`, `pop_front()`, `insert()` and `delete()`.

Q3. What does `$` mean in `q[$]` and in `int q [$]`? In the declaration it specifies that the array is a queue. In an index it stands for the index of the last element, `size()-1`.

Q4. How do you model a FIFO with a queue? A stack? FIFO: `push_back()` to write and `pop_front()` to read. Stack: `push_back()` to push and `pop_back()` to pop.

Q5. What is the difference between a queue and a dynamic array? A dynamic array must be allocated with `new[]` and is resized only by reallocating the whole array. A queue manages its own storage and can add or remove single elements at either end in constant time. Both can be indexed and both support the array methods.

Q6. What is a bounded queue? A queue with a maximum index, declared as `int q [$:N];`, which can hold at most N+1 elements. Elements that would go beyond the bound are discarded with a warning.

Q7. How do you delete the last element of a queue without using `pop_back()`? `q.delete(q.size() - 1);` or `q = q[0:$-1];`

Q8. How do you remove all elements of a queue? `q.delete();` or `q = {};`

Q9. Write three ways to append the value 5 to queue `q`. `q.push_back(5);`, `q = {q, 5};` and `q[$+1] = 5;`

Q10. Why are the results of `find()` returned as a queue? Because the number of matching elements is not known in advance. It may be zero, one or many, and a queue can hold any number.

Part 3: Associative Arrays and Choosing an Array Type

3.1 Associative Arrays

In plain English

Every array so far uses a small, contiguous range of integers as its index: 0, 1, 2, and so on. That fails in two situations.

1. **The index range is huge but only a few entries are used.** A design has a 32-bit address bus, so its memory has about four billion locations. A test writes to perhaps fifty of them. Declaring `logic [7:0] mem [4294967296]` would need gigabytes; the simulation would not even start.
2. **The natural index is not a number.** You want to look up a count by the *name* of an operation, or find a transaction by an ID, or store a setting under a string key.

An associative array solves both. It works like a dictionary or a phone book: you store a value under a **key**, and later look it up with the same key. Only the keys you actually store take up memory. The key can be a number, a string, or other types.

Technical definition

An associative array is an unpacked array whose index type is specified in the declaration, and whose elements are allocated individually when they are first written. It implements a lookup table from a **key** (index) to a **value** (element). Entries are kept in the sorted order of their keys, and the built-in methods traverse them in that order.

Other languages call this structure a map, a dictionary or a hash.

Syntax

```
// data_type  name [index_type];
int         count   [string];          // key: string,           value: int
logic [7:0] mem     [bit [31:0]];      // key: 32-bit address,   value: byte
bit [8:0]   exp     [int];             // key: int (signed),     value: 9-bit result
int         hist    [alu_op_e];        // key: an enum,          value: int
alu_txn     pending [int];             // key: int,              value: class handle
int         wild    [*];               // key: any integral value ("wildcard" index)

// initialisation with an assignment pattern of key : value pairs
int level [string] = '{"low": 1, "mid": 5, "high": 9};
int dflt  [string] = '{"a": 1, default: -1};    // default: value returned for missing keys
```

The wildcard form `[*]` accepts any integral key of any width but has restrictions (it cannot be used with `foreach`, and the traversal methods are awkward), so it is avoided in practice. Always name a specific index type.

Methods

Method	Prototype	What it does
`num()`	`function int num();`	Number of entries

Method	Prototype	What it does
`size()`	`function int size();`	Same as `num()`
`exists(key)`	`function int exists(input index);`	1 if an entry with this key exists, otherwise 0
`delete(key)`	`function void delete(input index);`	Removes that one entry. No effect if it does not exist
`delete()`	`function void delete();`	Removes all entries
`first(key)`	`function int first(ref index);`	Sets `key` to the **smallest** key. Returns 1, or 0 if the array is empty
`last(key)`	`function int last(ref index);`	Sets `key` to the **largest** key. Returns 1, or 0 if the array is empty
`next(key)`	`function int next(ref index);`	Sets `key` to the next key **after** its current value. Returns 1, or 0 if there is none (then `key` is unchanged)
`prev(key)`	`function int prev(ref index);`	Sets `key` to the previous key **before** its current value. Returns 1, or 0 if there is none (then `key` is unchanged)

The four traversal methods have an unusual shape that interviewers like to test:

- Their **argument is an output**. You pass a variable; the method writes a key into it.
- Their **return value is a status** (found or not found), not the key and not the value.

Reading, writing and existence

Action	Result
Write `aa[key] = v;` when the key does not exist	The entry is created
Write when the key exists	The value is replaced
Read `aa[key]` when the key exists	The stored value
Read `aa[key]` when the key does **not** exist	The default value of the element type (0, X, `""`, `null`), usually with a warning. **No entry is created**
`aa[key]++` or `aa[key] += n` when the key does not exist	The entry is created with the default value, then the operation is applied
`aa.exists(key)`	The only reliable way to ask whether an entry is present

If the declaration's initialiser contains `default: value`, reading a missing key returns that value without a warning.

Order of the keys

Entries are always traversed in sorted key order, regardless of the order in which they were written:

Index type	Order
Unsigned integral (`bit [31:0]`, `int unsigned`)	Numerically increasing
Signed integral (`int`, `integer`, `byte`)	Numerically increasing, **negative keys first**

Index type	Order
`string`	Lexicographic (dictionary) order by character code: `"B"` comes before `"a"`
Enum	By the numeric value of the members
Class handle	An order exists and is consistent within a run, but it is arbitrary

How it works internally

```
int score [string];

score["ravi"] = 82;
score["anu"]  = 95;
score["zoya"] = 71;
score["maya"] = 88;

What is stored: 4 separate (key, value) entries, kept sorted by key

     key        value
   +--------+--------+
   | "anu"  |   95   |   <- first()
   +--------+--------+
   | "maya" |   88   |
   +--------+--------+
   | "ravi" |   82   |
   +--------+--------+
   | "zoya" |   71   |   <- last()
   +--------+--------+

score["tom"]         -> not found : returns 0, nothing is added
score.exists("tom")  -> 0
score.num()          -> 4
```

The standard does not fix the data structure; simulators use a balanced tree or a hash table combined with ordering. What matters for you:

- Memory is proportional to the number of entries, not to the range of the keys.
- A lookup must search for the key, so it is slower than indexing a fixed or dynamic array.
- Entries do not have positions. There is no "element number 3"; there are only keys.

Complete example

tb_assoc_array.sv

```
module tb_assoc_array;

  int    score [string];
  int    level [int];
  string name;
  int    key;

  initial begin
    score["ravi"] = 82;
    score["anu"]  = 95;
    score["zoya"] = 71;
    score["maya"] = 88;
    $display("1. entries = %0d", score.num());

    if (score.exists("anu"))  $display("2. anu is present, score %0d", score["anu"]);
    if (!score.exists("tom")) $display("3. tom is not present");

    if (score.first(name))
      do $display("4.   %s = %0d", name, score[name]);
```

```
      while (score.next(name));

    void'(score.last(name));  $display("5. last key          = %s", name);
    void'(score.prev(name));  $display("6. key before zoya   = %s", name);

    score.delete("ravi");
    $display("7. after delete(\"ravi\") entries = %0d", score.num());

    foreach (score[k]) $display("8.   %s = %0d", k, score[k]);

    score["maya"]++;                      // existing entry
    score["neel"]++;                      // new entry: created as 0, then incremented
    $display("9. maya = %0d  neel = %0d  entries = %0d",
             score["maya"], score["neel"], score.num());

    // integer keys: stored in numeric order, negative first for a signed index
    level[5]   = 50;
    level[-3]  = 30;
    level[100] = 10;
    void'(level.first(key));  $display("10. first key = %0d", key);
    void'(level.next(key));   $display("11. next key  = %0d", key);
    void'(level.last(key));   $display("12. last key  = %0d", key);
    if (!level.next(key))     $display("13. no key after %0d", key);

    $display("14. level[7] = %0d, entries still %0d", level[7], level.num());

    score.delete();
    $display("15. after delete() entries = %0d", score.num());
  end

endmodule
```

EXPECTED OUTPUT

```
1. entries = 4
2. anu is present, score 95
3. tom is not present
4.   anu = 95
4.   maya = 88
4.   ravi = 82
4.   zoya = 71
5. last key          = zoya
6. key before zoya   = ravi
7. after delete("ravi") entries = 3
8.   anu = 95
8.   maya = 88
8.   zoya = 71
9. maya = 89  neel = 1  entries = 4
10. first key = -3
11. next key  = 5
12. last key  = 100
13. no key after 100
14. level[7] = 0, entries still 3
15. after delete() entries = 0
```

Line by line

Line	Explanation
1	Four writes created four entries. No `new` is needed.
2, 3	`exists()` tests for a key without reading the element.
4	The standard traversal idiom. `first(name)` puts the smallest key into `name` and returns 1 because the array is not empty. The `do` ... `while` body runs for that key, then `next(name)` moves `name` to the following key. When there is no further key, `next` returns 0 and the loop ends. The keys come out in dictionary order, not in the order written.
5, 6	`last()` gives the largest key. `prev()` steps backwards from the key currently in `name`. The return value is discarded with `void'()`.

Line	Explanation
7	`delete("ravi")` removes one entry.
8	`foreach` does the same traversal as line 4 with less code. The loop variable `k` automatically has the index type (`string`).
9	Incrementing an existing entry changes it from 88 to 89. Incrementing a missing entry creates it with the default value 0 and then adds 1.
10 to 12	With a signed index type the keys are ordered -3, 5, 100.
13	`next()` returns 0 when there is no later key, and leaves `key` unchanged.
14	Reading a missing key returns the default value (0) and does **not** create an entry. A simulator warning is normal here.
15	`delete()` with no argument removes everything.

A practical example: sparse memory model

tb_sparse_memory.sv

```
module tb_sparse_memory;

  logic [7:0] mem [bit [31:0]];          // only the written addresses use memory

  function automatic void mem_write(bit [31:0] addr, logic [7:0] data);
    mem[addr] = data;
  endfunction

  function automatic logic [7:0] mem_read(bit [31:0] addr);
    if (mem.exists(addr)) return mem[addr];
    return 'x;                           // unwritten memory reads as unknown
  endfunction

  initial begin
    mem_write(32'h0000_0010, 8'hAA);
    mem_write(32'hFFFF_FFF0, 8'hBB);
    mem_write(32'h8000_0000, 8'hCC);
    mem_write(32'h0000_0010, 8'hDD);     // overwrite

    $display("read 00000010 = %h", mem_read(32'h0000_0010));
    $display("read 00000020 = %h", mem_read(32'h0000_0020));
    $display("locations in use = %0d", mem.num());
    foreach (mem[a]) $display("  mem[%h] = %h", a, mem[a]);
  end

endmodule
```

EXPECTED OUTPUT

```
read 00000010 = dd
read 00000020 = xx
locations in use = 3
  mem[00000010] = dd
  mem[80000000] = cc
  mem[fffffff0] = bb
```

Explanation

Code	Explanation
`logic [7:0] mem [bit [31:0]];`	The key is an unsigned 32-bit address. Four billion possible addresses, three entries stored.

Code	Explanation
`mem_write`	Writing creates or replaces the entry. The second write to address `10` replaces `AA` with `DD`.
`mem.exists(addr)`	A real memory that was never written contains unknown data, so the model returns X for a missing address instead of the silent default. This lets the scoreboard detect reads of uninitialised memory.
`foreach (mem[a])`	Visits the addresses in increasing unsigned order. Had the index type been `int` (signed), `80000000` and `FFFFFFF0` would be negative and would come first.

A practical example: counting by key

tb_assoc_histogram.sv

```
module tb_assoc_histogram;

  typedef enum logic [2:0] {
    ALU_ADD, ALU_SUB, ALU_AND, ALU_OR, ALU_XOR, ALU_NOT, ALU_SHL, ALU_SHR
  } alu_op_e;

  alu_op_e ops [10] = '{ALU_ADD, ALU_SUB, ALU_ADD, ALU_XOR, ALU_ADD,
                        ALU_SHL, ALU_SUB, ALU_ADD, ALU_XOR, ALU_ADD};
  int      hist [alu_op_e];

  initial begin
    foreach (ops[i]) hist[ops[i]]++;

    $display("%0d different operations seen", hist.num());
    foreach (hist[op]) $display("  %-8s : %0d", op.name(), hist[op]);

    if (!hist.exists(ALU_NOT)) $display("ALU_NOT was never generated");
  end

endmodule
```

EXPECTED OUTPUT

```
4 different operations seen
  ALU_ADD  : 5
  ALU_SUB  : 2
  ALU_XOR  : 2
  ALU_SHL  : 1
ALU_NOT was never generated
```

The array is indexed by the enum. `hist[ops[i]]++` creates an entry the first time an operation is seen and increments it afterwards. Traversal is in the order of the enum values (`ALU_ADD` = 0, `ALU_SUB` = 1, `ALU_XOR` = 4, `ALU_SHL` = 6). The `exists()` test at the end is a hand-made version of a coverage hole report; Module 10 shows how covergroups do this properly.

A practical example: out-of-order scoreboard

When a DUT may return responses in a different order from the requests, a queue is the wrong structure, because `pop_front()` assumes order. Expected results are instead stored in an associative array keyed by a transaction ID.

tb_assoc_scoreboard.sv

```
module tb_assoc_scoreboard;

  logic [8:0] exp_by_id [int];

  function automatic void check(int id, logic [8:0] actual);
    if (!exp_by_id.exists(id)) begin
      $display("FAIL: id %0d was not expected", id);
      return;
    end
    if (actual !== exp_by_id[id])
      $display("FAIL: id %0d actual=%0d expected=%0d", id, actual, exp_by_id[id]);
    else
      $display("PASS: id %0d = %0d", id, actual);
    exp_by_id.delete(id);                  // this expectation has been consumed
  endfunction

  initial begin
    exp_by_id[1] = 9'd30;
    exp_by_id[2] = 9'd40;
    exp_by_id[3] = 9'd50;

    check(3, 9'd50);                       // responses arrive out of order
    check(1, 9'd31);
    check(7, 9'd0);

    $display("still pending: %0d", exp_by_id.num());
    foreach (exp_by_id[id]) $display("  no response received for id %0d", id);
  end

endmodule
```

EXPECTED OUTPUT

```
PASS: id 3 = 50
FAIL: id 1 actual=31 expected=30
FAIL: id 7 was not expected
still pending: 1
  no response received for id 2
```

Each response is looked up by its ID, compared, and removed. Whatever is left at the end of the test is a request that never received a response.

Associative array versus the other kinds

Property	Dynamic array	Queue	Associative array
Index	0 to size-1	0 to size-1	Any declared type; any values
Elements contiguous	Yes	Yes	No (sparse)
Allocation	`new[n]`	Automatic	Automatic, one entry per written key
Access speed	Fastest	Fast	Slower (a search per access)
Memory	All elements	All elements	Only the entries that exist
Find out if an element exists	Compare index with `size()`	Compare index with `size()`	`exists(key)`
Number of elements	`size()`	`size()`	`num()` or `size()`
Traversal	`foreach`, `for`	`foreach`, `for`	`foreach`, or `first` / `next`, `last` / `prev`

Property	Dynamic array	Queue	Associative array
`sort`, `rsort`, `reverse`, `shuffle`	Yes	Yes	**No**
`find`, `min`, `max`, `unique`, `sum`...	Yes	Yes	Yes
Literal	`'{1, 2, 3}`	`{1, 2, 3}`	`'{key: value, ...}`

Interview trap: "What does `aa.first(k)` return?" It returns 1 or 0: whether the array has any entry. The first key is delivered through the argument `k`. Writing `k = aa.first();` is wrong on both counts.

Interview trap: "How do you check whether a key is in an associative array?" With `exists()`. Testing `if (aa[key] == 0)` is wrong: a missing key and a key whose stored value is 0 look the same, and the read produces a warning.

Common mistake: Expecting entries to come back in insertion order. They always come back in sorted key order. If insertion order matters, also keep a queue of the keys.

Common mistake: Using a signed index type for addresses. With `mem [int]`, the address `32'h8000_0000` is a negative key and sorts before address 0. Use `bit [31:0]` or `int unsigned` for addresses.

Common mistake: Deleting entries of the same associative array inside a `foreach` over it. Changing the set of keys while iterating gives undefined results. Collect the keys to delete in a queue first, then delete them in a second loop.

Verification application: Associative arrays are the standard choice for (1) memory models of RAM, ROM and register spaces; (2) out-of-order scoreboards keyed by ID or address; (3) per-key statistics, such as transactions per address or errors per message string; (4) configuration databases that map a string name to a value (the UVM configuration database is built this way); (5) address-to-register-object maps in register models.

Interview questions

Q1. What is an associative array? An array whose elements are allocated individually when written and are accessed by a key of a declared index type. It is a sparse lookup table from key to value.

Q2. When would you use an associative array instead of a dynamic array? When the index range is very large but sparsely used, such as a memory model with a 32-bit address, or when the index is not a small contiguous integer, such as a string or an ID.

Q3. What index types can an associative array have? Any integral type, `string`, an enum, a class handle, a packed struct, or the wildcard `*` for any integral value.

Q4. List the associative array methods. `num()`, `size()`, `exists()`, `delete()`, `first()`, `last()`, `next()`, `prev()`.

Q5. What do `first()` and `next()` return, and how is the key obtained? They return 1 if a key was found and 0 otherwise. The key is written into the variable passed as the argument, which must be of the index type.

Q6. What happens when you read a key that does not exist? The default value of the element type is returned, a warning is normally issued, and no entry is created.

Q7. In what order does `foreach` visit an associative array? In increasing order of the keys: numeric order for integral keys (signed keys with negatives first) and lexicographic order for string keys.

Q8. What is the difference between `num()` and `size()` for an associative array? None. Both return the number of entries.

Q9. Can you sort an associative array? No. `sort()`, `rsort()`, `reverse()` and `shuffle()` are not available, because the entries are always kept in key order. To get the values in sorted order, copy them into a queue and sort the queue.

Q10. How do you delete one entry? All entries? `aa.delete(key);` and `aa.delete();`

Q11. Write code to print all keys and values of `int aa [string]` without using `foreach`.

```
string k;
if (aa.first(k))
  do $display("%s = %0d", k, aa[k]);
  while (aa.next(k));
```

Q12. How would you model a 4 GB memory in a testbench? `logic [7:0] mem [bit [31:0]];`. Only written locations consume memory. A read function returns X (or a defined default) when `exists(addr)` is false.

3.2 Choosing the Right Array Type

All array kinds side by side

Property	Packed	Fixed-size	Dynamic	Queue	Associative
Declaration	`bit [7:0] p;`	`int a [8];`	`int d [];`	`int q [$];`	`int aa [key_t];`
Size set	Compile time	Compile time	Run time, `new[]`	Automatic	Automatic
Resizable	No	No	Yes, by reallocation	Yes, any time	Yes, per entry
Index	Integer range	Integer range	0..N-1	0..N-1	Any type, sparse
Element type	Bits only	Any	Any	Any	Any
Is one number	Yes	No	No	No	No
Add/remove single elements	No	No	Costly	Cheap at the ends	Cheap by key
Indexed access speed	Fastest	Fastest	Fast	Fast	Slowest
Memory use	Minimal	Full size	Current size	Current size	Entries only, plus overhead per entry

Property	Packed	Fixed-size	Dynamic	Queue	Associative
Element count	`$size`	`$size`	`size()`	`size()`	`num()`
Array methods	No	Yes	Yes	Yes	Locator and reduction only
Randomizable size	No	No	Yes	Yes	No
Synthesizable	Yes	Yes	No	No	No

A decision procedure

```
Is it a bus, a register or a bit field that must behave as one number?
     yes -> packed array                      logic [31:0] data;

Is the number of elements fixed by the design or known when writing the code?
     yes -> fixed-size array                  logic [31:0] regs [32];

Is the index sparse, very large, or not an integer (string, ID, address)?
     yes -> associative array                 logic [7:0] mem [bit [31:0]];

Are elements added and removed one at a time as the simulation runs?
     yes -> queue                             alu_txn exp_q [$];

Otherwise (size chosen at run time, then used as a block)
         -> dynamic array                     rand bit [7:0] payload [];
```

Where each kind appears in a verification environment

Testbench need	Array	Example declaration
DUT data bus, register value	Packed	`logic [31:0] rdata;`
Model of a small register file	Fixed-size	`logic [31:0] regs [32];`
One virtual interface per DUT port	Fixed-size	`virtual alu_if vif [4];`
Random-length packet payload	Dynamic	`rand bit [7:0] payload [];`
Data loaded from a file whose length is read first	Dynamic	`bit [31:0] image [];`
Expected results, in-order checking	Queue	`alu_txn exp_q [$];`
Reference model of a FIFO	Queue	`bit [7:0] fifo_q [$];`
List of all transactions seen, for a final report	Queue	`alu_txn history [$];`
Results of a search (`find`)	Queue	`int idx_q [$];`
Memory model	Associative	`logic [7:0] mem [bit [31:0]];`
Expected results, out-of-order checking	Associative	`alu_txn exp_by_id [int];`
Count of events per name	Associative	`int err_count [string];`
One expected queue per port	Fixed array of queues	`alu_txn exp_q [4][$];`
One expected queue per ID	Associative array of queues	`alu_txn exp_q [int][$];`

> **How to say it in an interview:** "I choose the array type from how the data is accessed. A fixed-size array when the size is a design constant. A dynamic array when the size is decided at run time but then stable, for example a randomized payload. A queue when I continuously push and pop, for example the scoreboard's expected list for in-order checking. An associative array when the index is sparse or not an integer, for example a memory model or an out-of-order scoreboard keyed by transaction ID."

Interview questions

Q1. Which array type would you use for a scoreboard, and why? A queue when the DUT preserves order: expected transactions are pushed at the back and popped from the front. An associative array keyed by ID when responses can arrive out of order, because a response must be found by its key and not by its position.

Q2. Which array type would you use to model a memory with a 32-bit address? An associative array indexed by `bit [31:0]`, because only a tiny fraction of the address space is used.

Q3. Which array types can have their size randomized? Dynamic arrays and queues, by constraining `size()`. Fixed-size arrays have a constant size, and the size of an associative array cannot be randomized.

Q4. Which is faster to index, a dynamic array or an associative array? A dynamic array. Its elements are contiguous, so the address of an element is computed directly. An associative array must search for the key.

Q5. You need to store 1000 transactions and frequently delete the 500th. Which array? If the transactions are identified by a key such as an ID, an associative array, because deletion by key does not move other entries. A queue would work but each `delete(index)` moves all later elements.

Q6. How do you declare an array of 8 queues of bytes? `bit [7:0] q [8][$];`

Part 4: Array Methods: Searching, Ordering and Reduction

4.1 Overview of the Built-in Array Methods

In plain English

Finding an element, sorting, adding everything up: these jobs come up constantly, and each would normally need a loop. SystemVerilog provides them as built-in methods, so that one line replaces five. The methods are called on the array with the dot operator, for example `arr.sort()` or `arr.sum()`.

The three families

Family	Methods	What they do	Return
Locator (searching)	`find`, `find_index`, `find_first`, `find_first_index`, `find_last`, `find_last_index`, `min`, `max`, `unique`, `unique_index`	Search the array for elements or indices	Always a **queue**
Ordering	`sort`, `rsort`, `reverse`, `shuffle`	Rearrange the elements **in place**	Nothing (`void`)
Reduction	`sum`, `product`, `and`, `or`, `xor`	Combine all elements into one value	A single value

Which arrays support which methods

Method family	Fixed-size	Dynamic	Queue	Associative	Packed
Locator	Yes	Yes	Yes	Yes	No
Ordering	Yes	Yes	Yes	**No**	No
Reduction	Yes	Yes	Yes	Yes	No

All of these methods are simulation-only testbench features.

The with clause and the iterator

Most methods accept a `with` clause: an expression in parentheses that is evaluated once for every element. Inside the expression, the current element is called `item`.

```
r = arr.find with (item > 5);          // keep the elements for which the expression is true
```

Feature	Syntax	Meaning
Default iterator name	`item`	The current element
Your own iterator name	`arr.find(x) with (x > 5)`	The name in the parentheses after the method replaces `item`

Feature	Syntax	Meaning
Index of the current element	`item.index`	The index of the element being examined (`x.index` with a custom name)
Member access	`item.addr` , `item.len`	When the elements are structs or class objects

What the `with` expression means depends on the family:

Family	Meaning of the `with` expression	Required?
`find...` methods	A condition: the element is selected if the expression is true	**Mandatory**
`min` , `max` , `unique` , `unique_index`	The value to compare instead of the element itself	Optional
`sort` , `rsort`	The key to sort by	Optional
`reverse` , `shuffle`	Not allowed	Not allowed
Reduction methods	The value to combine instead of the element itself	Optional

`item.index` is part of the standard since IEEE 1800-2009; a few older tool versions do not support it.

4.2 Array Searching: Locator Methods

Technical definition

Locator methods traverse an unpacked array and return a **queue** containing either the matching elements or their indices. The queue is empty when nothing matches.

Method	Returns a queue containing
`find() with (expr)`	**All elements** for which `expr` is true
`find_index() with (expr)`	The **indices** of all elements for which `expr` is true
`find_first() with (expr)`	The **first** element for which `expr` is true (0 or 1 entries)
`find_first_index() with (expr)`	The index of the first such element (0 or 1 entries)
`find_last() with (expr)`	The **last** element for which `expr` is true (0 or 1 entries)
`find_last_index() with (expr)`	The index of the last such element (0 or 1 entries)
`min()`	The element with the smallest value (1 entry; empty if the array is empty)
`max()`	The element with the largest value (1 entry; empty if the array is empty)
`unique()`	One element for each distinct value
`unique_index()`	One index for each distinct value

Type of the result

Method returns	Result must be declared as
Elements (`find` , `find_first` , `find_last` , `min` , `max` , `unique`)	A queue of the **element type**: `int r [$];` for an array of `int`

Method returns	Result must be declared as
Indices (`find_index` , `find_first_index` , `find_last_index` , `unique_index`)	A queue of `int` for fixed, dynamic and queue arrays; a queue of the **index type** for an associative array

Why results are queues

The number of matches is unknown until run time: none, one or many. Even the methods that can return at most one element (`find_first` , `min` , `max`) return a queue, because "no match" must be representable. You therefore always check `size()` before using `result[0]` .

How it works internally

```
int d [] = '{4, 9, 2, 9, 7, 2, 5};
index :       0  1  2  3  4  5  6

d.find with (item > 5)

   index 0 : item = 4   4 > 5 ?  no
   index 1 : item = 9   9 > 5 ?  yes  -> result = {9}
   index 2 : item = 2            no
   index 3 : item = 9            yes  -> result = {9, 9}
   index 4 : item = 7            yes  -> result = {9, 9, 7}
   index 5 : item = 2            no
   index 6 : item = 5            no
                                        returns {9, 9, 7}

d.find_index with (item > 5)             returns {1, 3, 4}
d.find_first with (item > 5)             stops at index 1,   returns {9}
d.find_last_index with (item < 5)        scans from the end, returns {5}
```

Each method is a linear scan: its cost is proportional to the number of elements.

Complete example

tb_array_locator.sv

```
module tb_array_locator;

  int d [] = '{4, 9, 2, 9, 7, 2, 5};
  int r  [$];          // for returned elements
  int ix [$];          // for returned indices

  initial begin
    r  = d.find with (item > 5);              $display("1.  find > 5             : %p", r);
    ix = d.find_index with (item > 5);        $display("2.  find_index > 5       : %p", ix);
    r  = d.find_first with (item > 5);        $display("3.  find_first > 5       : %p", r);
    ix = d.find_first_index with (item > 5);  $display("4.  find_first_index > 5 : %p", ix);
    r  = d.find_last with (item < 5);         $display("5.  find_last < 5        : %p", r);
    ix = d.find_last_index with (item < 5);   $display("6.  find_last_index < 5  : %p", ix);

    r  = d.find with (item > 100);
    $display("7.  find > 100           : size %0d", r.size());

    r  = d.min();                             $display("8.  min                  : %p", r);
    r  = d.max();                             $display("9.  max                  : %p", r);

    r  = d.unique();
    r.sort();                                 $display("10. unique (sorted)      : %p", r);
    ix = d.unique_index();
    $display("11. unique_index         : size %0d", ix.size());

    r  = d.find with (item.index >= 4);       $display("12. index >= 4           : %p", r);
    r  = d.find(x) with (x % 2 == 0);         $display("13. even values          : %p", r);

    ix = d.find_index with (item == 9);
    $display("14. the value 9 occurs %0d times, first at index %0d", ix.size(), ix[0]);
  end

endmodule
```

EXPECTED OUTPUT

```
1.  find > 5             : '{9, 9, 7}
2.  find_index > 5       : '{1, 3, 4}
3.  find_first > 5       : '{9}
4.  find_first_index > 5 : '{1}
5.  find_last < 5        : '{2}
6.  find_last_index < 5  : '{5}
7.  find > 100           : size 0
8.  min                  : '{2}
9.  max                  : '{9}
10. unique (sorted)      : '{2, 4, 5, 7, 9}
11. unique_index         : size 5
12. index >= 4           : '{7, 2, 5}
13. even values          : '{4, 2, 2}
14. the value 9 occurs 2 times, first at index 1
```

Line by line

Line	Explanation
1	All elements greater than 5, in array order.
2	Their indices. Note the result variable is the `int` queue `ix`.
3, 4	Only the first match and its index. The result is still a queue, with one entry.
5, 6	Elements less than 5 are at indices 0 (4), 2 (2) and 5 (2). The last one is index 5, value 2.
7	Nothing matches, so the queue is empty. Reading `r[0]` here would return 0 with a warning.

Line	Explanation
8, 9	`min()` and `max()` return one-element queues. To get the value, use `r[0]`.
10	The array contains five distinct values. The standard does not define the order in which `unique()` returns them (most simulators return them in order of first appearance), so the result is sorted before printing to make the output predictable.
11	Five indices, one per distinct value. For a duplicated value the standard does not say which of its indices is returned, so only the count is printed.
12	`item.index` gives access to the position: elements at indices 4, 5 and 6.
13	A custom iterator name `x` is used instead of `item`.
14	A common idiom: the size of `find_index` is the number of occurrences. `ix` is `{1, 3}`.

Searching arrays of structs or objects

The `with` clause becomes more useful when the elements have members. The same code works unchanged when the elements are class handles such as `alu_txn` (Module 4).

tb_array_locator_struct.sv

```
module tb_array_locator_struct;

  typedef struct {
    int       id;
    bit [7:0] addr;
    int       len;
  } pkt_t;

  pkt_t pkts [$];
  pkt_t res  [$];
  int   idx  [$];

  initial begin
    pkts = '{ '{1, 8'h40, 12}, '{2, 8'h10, 64}, '{3, 8'h40, 3}, '{4, 8'h80, 30} };

    res = pkts.find with (item.addr == 8'h40);
    foreach (res[i]) $display("1. addr 40 : id %0d", res[i].id);

    res = pkts.max with (item.len);
    $display("2. longest packet  : id %0d (len %0d)", res[0].id, res[0].len);

    res = pkts.min with (item.len);
    $display("3. shortest packet : id %0d (len %0d)", res[0].id, res[0].len);

    idx = pkts.find_first_index(p) with (p.len > 20);
    $display("4. first packet longer than 20 is at index %0d", idx[0]);

    res = pkts.unique with (item.addr);
    $display("5. %0d different addresses", res.size());

    idx = pkts.find_index with (item.len > 100);
    if (idx.size() == 0) $display("6. no packet longer than 100");
  end

endmodule
```

EXPECTED OUTPUT

```
1. addr 40 : id 1
1. addr 40 : id 3
2. longest packet  : id 2 (len 64)
3. shortest packet : id 3 (len 3)
4. first packet longer than 20 is at index 1
5. 3 different addresses
6. no packet longer than 100
```

Line	Explanation
1	`find` returns the whole elements (structs) whose `addr` is `40`: the packets with id 1 and 3.
2, 3	With a `with` clause, `max` and `min` compare the given expression (`len`) but return the **element**, so its other members are available.
4	A custom iterator name `p`. Packet 2 (index 1, len 64) is the first with `len` above 20.
5	`unique with (item.addr)` keeps one element per distinct address: `40`, `10`, `80`.
6	Always test `size()` before using the result of a locator method.

Useful one-line idioms

Task	Code
Does the array contain the value `v`?	`if (v inside {arr})`
How many elements satisfy a condition?	`n = arr.find with (item > 5).size();` or `n = arr.sum() with (int'(item > 5));`
Index of the largest element	`mx = arr.max(); ix = arr.find_first_index with (item == mx[0]);`
Remove duplicates	`q = q.unique();`
Are all elements different?	`arr.unique().size() == arr.size()`
Remove all elements that satisfy a condition	`q = q.find with (!(item > 5));`
Is the array already sorted?	`tmp = arr; tmp.sort(); if (tmp == arr) ...`

Calling a method directly on the result of another (`arr.find with (...).size()`) is accepted by current simulators; if a tool rejects it, store the result in a queue first.

> **Interview trap:** "What does `find_first()` return?" A queue, not a single element. `int x = arr.find_first with (item > 5);` is a compile error (a queue cannot be assigned to an `int`). Declare `int r [$];` and use `r[0]` after checking `r.size()`.

> **Interview trap:** "Is the `with` clause optional for `find()`?" No. For all six `find...` methods it is mandatory, because without a condition there is nothing to search for. It is optional for `min`, `max`, `unique` and `unique_index`.

> **Common mistake:** Declaring the result of `find_index` with the element type. Indices are `int` (or the index type for an associative array), whatever the element type is.

> **Common mistake:** Using `min()` or `max()` on an empty array and reading `[0]` of the empty result.

> **Verification application:** Locator methods are used in scoreboards and checkers. To match a response against pending requests: `idx = pending.find_first_index with (item.id == rsp.id);` and then `pending.delete(idx[0]);`. To check that a randomized array of IDs has no duplicates: `assert (ids.unique().size() == ids.size());`. To report the worst case at the end of a test: `worst = latency_q.max();`.

Interview questions

Q1. What do the locator methods return? A queue: of elements for `find`, `find_first`, `find_last`, `min`, `max` and `unique`; of indices for `find_index`, `find_first_index`, `find_last_index` and `unique_index`.

Q2. What is the difference between `find()` and `find_index()`? `find()` returns the matching elements. `find_index()` returns the positions of the matching elements.

Q3. What is the difference between `find_first()` and `find()`? `find()` returns every match. `find_first()` stops at the first match and returns a queue with at most one element.

Q4. What is `item` in a `with` clause? The default name of the iterator: it refers to the array element currently being examined. A different name can be given as an argument of the method, and `item.index` gives the element's index.

Q5. How do you find the number of elements greater than 10 in an array? `int r [$] = arr.find with (item > 10);` then `r.size()`. Alternatively `arr.sum() with (int'(item > 10))`.

Q6. What do `min()` and `max()` return for an empty array? An empty queue.

Q7. How do you find the index of the maximum element? Get the maximum with `m = arr.max();`, then `ix = arr.find_first_index with (item == m[0]);`.

Q8. How do you remove duplicate values from a queue? `q = q.unique();`

Q9. What does `unique_index()` return? A queue of indices, one for each distinct value in the array.

Q10. For an associative array `int aa [string]`, what is the type of the result of `aa.find_index with (item > 5)`? A queue of `string`, because the index type is `string`.

4.3 Array Sorting and Ordering Methods

Technical definition

Ordering methods rearrange the elements of the array itself. They return nothing. They work on fixed-size arrays, dynamic arrays and queues, but not on associative arrays.

Method	Effect	`with` clause
`sort()`	Sorts in **ascending** order	Optional: the key to sort by
`rsort()`	Sorts in **descending** order	Optional: the key to sort by
`reverse()`	Reverses the current order of the elements	Not allowed

Method	Effect	`with` clause
`shuffle()`	Puts the elements in a random order	Not allowed

Elements are compared according to their type: integers by value (signed types as signed numbers), strings in lexicographic order. For elements that cannot be compared directly, such as structs and class handles, a `with` clause naming a sortable member is required.

How it works internally

```
int d [] = '{4, 9, 2, 9, 7, 2, 5};

d.sort();      d = '{2, 2, 4, 5, 7, 9, 9}       ascending, in place
d.rsort();     d = '{9, 9, 7, 5, 4, 2, 2}       descending, in place
d.reverse();   d = '{2, 2, 4, 5, 7, 9, 9}       mirror of the previous order
d.shuffle();   d = some random permutation      same elements, random order

With a key:
pkts.sort with (item.len);     the elements are whole structs,
                               but they are ordered by their len member
```

The original order is lost. If you need both the original and the sorted version, copy the array first and sort the copy. The standard does not say what happens to the relative order of elements with equal keys, so do not rely on it.

Complete example

tb_array_ordering.sv

```
module tb_array_ordering;

  typedef struct {
    int id;
    int len;
  } pkt_t;

  int    d [] = '{4, 9, 2, 9, 7, 2, 5};
  int    copy [];
  string names [$] = {"ravi", "anu", "zoya", "maya"};
  pkt_t  pkts [4] = '{ '{1, 12}, '{2, 64}, '{3, 3}, '{4, 30} };

  initial begin
    copy = d;                       // keep the original order
    d.sort();                       $display("1. sort     : %p", d);
    d.rsort();                      $display("2. rsort    : %p", d);
    d.reverse();                    $display("3. reverse  : %p", d);
    $display("4. original is untouched in the copy : %p", copy);

    names.sort();                   $display("5. strings sorted  : %p", names);
    names.rsort();                  $display("6. strings rsorted : %p", names);

    pkts.sort with (item.len);
    $write("7. packets by len        :");
    foreach (pkts[i]) $write(" id%0d(len %0d)", pkts[i].id, pkts[i].len);
    $display("");

    pkts.rsort(p) with (p.id);
    $write("8. packets by id, reverse:");
    foreach (pkts[i]) $write(" id%0d", pkts[i].id);
    $display("");

    d.sort with (-item);            $display("9. sort with (-item) : %p", d);

    d.shuffle();
    $display("10. after shuffle: size %0d, sum %0d", d.size(), d.sum());
  end

endmodule
```

EXPECTED OUTPUT

```
1. sort     : '{2, 2, 4, 5, 7, 9, 9}
2. rsort    : '{9, 9, 7, 5, 4, 2, 2}
3. reverse  : '{2, 2, 4, 5, 7, 9, 9}
4. original is untouched in the copy : '{4, 9, 2, 9, 7, 2, 5}
5. strings sorted  : '{"anu", "maya", "ravi", "zoya"}
6. strings rsorted : '{"zoya", "ravi", "maya", "anu"}
7. packets by len        : id3(len 3) id1(len 12) id4(len 30) id2(len 64)
8. packets by id, reverse: id4 id3 id2 id1
9. sort with (-item) : '{9, 9, 7, 5, 4, 2, 2}
10. after shuffle: size 7, sum 38
```

Line by line

Line	Explanation
`copy = d;`	Ordering methods modify the array. The copy preserves the original.
1	Ascending order.
2	Descending order.

Line	Explanation
3	`reverse()` does not sort; it mirrors whatever order the array currently has. Since the array was in descending order, the result is ascending.
5, 6	Strings are compared character by character.
7	A struct cannot be compared as a whole, so the key is given with `with (item.len)`. The elements that move are the complete structs.
8	`rsort` with a custom iterator name, descending by `id`.
9	Sorting by the negated value gives descending order: an alternative to `rsort()`.
10	`shuffle()` produces a random permutation, so the order differs from run to run and between simulators. What can be stated for certain is that the same seven elements are still there: the size is 7 and the sum is 38.

sort, rsort, reverse and shuffle compared

Method	Result depends on current order?	Result predictable?	Typical use
`sort()`	No	Yes	Ordering results for reports, comparing two collections regardless of order
`rsort()`	No	Yes	Highest first: "top N" reports
`reverse()`	Yes	Yes	Reversing byte order of a payload; processing last-to-first
`shuffle()`	No	No (random)	Randomising the order of a fixed set of items, such as a list of addresses or tests

Interview trap: "What is the difference between `rsort()` and `reverse()`?" `rsort()` sorts into descending order whatever the starting order. `reverse()` only flips the existing order and does not compare elements at all. They give the same result only if the array was already sorted ascending.

Interview trap: "Can you sort an associative array?" No. Ordering methods are not defined for associative arrays, whose entries are always kept in key order.

Common mistake: Writing `sorted = arr.sort();`. Ordering methods return `void` and change `arr` itself. Write `sorted = arr; sorted.sort();`.

Common mistake: Calling `sort()` on an array of structs or class handles without a `with` clause. There is no defined ordering for such elements; give the key explicitly.

Verification application: (1) A scoreboard for a DUT that may reorder data compares sorted copies: `exp.sort(); act.sort(); if (exp != act) $error(...)`. (2) `shuffle()` is a simple way to visit every item of a list exactly once in random order, for example to write all registers in a random sequence: fill a queue with the addresses, shuffle it, then loop. (3) Sorting a queue of latencies gives the minimum, median and maximum for a performance report.

Interview questions

Q1. Which array ordering methods exist? `sort()`, `rsort()`, `reverse()` and `shuffle()`.

Q2. Do ordering methods return a new array? No. They return nothing and modify the array in place.

Q3. How do you sort an array of transactions by their address field? `txn_q.sort with (item.addr);`

Q4. How do you sort in descending order? `arr.rsort();`, or `arr.sort(); arr.reverse();`, or `arr.sort with (-item);` for signed numeric elements.

Q5. How do you sort by two fields, first by address and then by ID? Use a `with` expression that combines them with the more important field in the upper bits: `q.sort with ({item.addr, item.id});`. Both fields must be unsigned (or non-negative) for the concatenation to order correctly.

Q6. What does `shuffle()` do, and is its result repeatable? It rearranges the elements randomly. Like other random features, it is repeatable for the same simulator and the same seed, and differs between seeds.

Q7. Which array types support `sort()`? Fixed-size arrays, dynamic arrays and queues. Not associative arrays, and not packed arrays.

4.4 Array Reduction Methods

Technical definition

A reduction method combines all the elements of an unpacked array into a single value by applying one operator across them.

Method	Operation	Result for `'{1, 2, 3, 4}`
`sum()`	Adds all elements	10
`product()`	Multiplies all elements	24
`and()`	Bitwise AND of all elements	0
`or()`	Bitwise OR of all elements	7
`xor()`	Bitwise XOR of all elements	4

An optional `with` clause supplies an expression to be combined instead of the element itself:

```
total_len = pkts.sum() with (item.len);            // add up a member
sq        = arr.sum() with (item * item);          // sum of squares
evens     = arr.sum() with (item % 2 == 0 ? item : 0);   // sum of the even values only
```

The rule that causes the most bugs

> **Key point:** The result of a reduction method has **the type of the array element** (or the type of the `with` expression, if there is one). The calculation is done in that width. The width of the variable you assign the result to does not matter.

```
bit [7:0] b [] = '{200, 100, 50};

int total = b.sum();
     |
     |   elements are 8 bits  ->  the sum is computed in 8 bits
     |   200 + 100 + 50 = 350 = 1_0101_1110  ->  truncated to 0101_1110 = 94
     v
total = 94          (NOT 350, even though total is 32 bits wide)

int total = b.sum() with (int'(item));
     |
     |   the with expression is an int  ->  the sum is computed in 32 bits
     v
total = 350
```

The same rule produces a second classic bug. A comparison such as `item > 2` is **one bit** wide, so `arr.sum() with (item > 2)` adds up 1-bit values in a 1-bit result. Cast the comparison to `int` to count.

Complete example

tb_array_reduction.sv

```
module tb_array_reduction;

  int       n [] = '{1, 2, 3, 4};
  bit [7:0] b [] = '{200, 100, 50};
  bit [3:0] m [] = '{4'b1100, 4'b1010, 4'b1001};
  int       e [$];                          // empty queue
  int       total;

  initial begin
    $display("1. sum = %0d   product = %0d", n.sum(), n.product());
    $display("2. and = %0d   or = %0d   xor = %0d", n.and(), n.or(), n.xor());

    total = b.sum();
    $display("3. b.sum()                    = %0d", total);
    total = b.sum() with (int'(item));
    $display("4. b.sum() with (int'(item))  = %0d", total);

    $display("5. count of n > 2 : wrong = %0d   right = %0d",
             n.sum() with (item > 2), n.sum() with (int'(item > 2)));

    $display("6. and = %b   or = %b   xor = %b", m.and(), m.or(), m.xor());

    $display("7. sum of squares     = %0d", n.sum() with (item * item));
    $display("8. sum of even values = %0d", n.sum() with (item % 2 == 0 ? item : 0));
    $display("9. sum of index*value = %0d", n.sum() with (item.index * item));

    $display("10. sum of an empty queue = %0d", e.sum());
    $display("11. average = %0.2f", real'(n.sum()) / n.size());
  end

endmodule
```

EXPECTED OUTPUT

```
1. sum = 10   product = 24
2. and = 0   or = 7   xor = 4
3. b.sum()                    = 94
4. b.sum() with (int'(item))  = 350
5. count of n > 2 : wrong = 0   right = 2
6. and = 1000   or = 1111   xor = 1111
7. sum of squares     = 30
8. sum of even values = 6
9. sum of index*value = 20
10. sum of an empty queue = 0
11. average = 2.50
```

Line by line

Line	Explanation
1	1+2+3+4 = 10 and 1×2×3×4 = 24.
2	In binary the values are `001`, `010`, `011`, `100`. AND of all: `000`. OR of all: `111` = 7. XOR: `001^010` = `011`, `^011` = `000`, `^100` = `100` = 4.
3	The elements are 8 bits wide, so the sum is 8 bits wide: 350 mod 256 = 94. The 32-bit `total` receives the already truncated value.
4	Casting each element to `int` in the `with` clause makes the sum 32 bits wide: 350.
5	Two elements (3 and 4) are greater than 2. Without the cast each comparison is a 1-bit value and so is the sum: 1 + 1 = 2 does not fit in one bit and becomes 0. With `int'()` the count is correct.
6	Bit by bit over `1100`, `1010`, `1001`: AND is `1000`, OR is `1111`, XOR is `1111`.
7	1 + 4 + 9 + 16 = 30.
8	The conditional expression contributes the value for even elements and 0 otherwise: 2 + 4 = 6.
9	`item.index` is the position: 0×1 + 1×2 + 2×3 + 3×4 = 20.
10	Reducing an empty array gives 0 for `sum()`.
11	`n.sum() / n.size()` would be the integer division 10 / 4 = 2. Casting to `real` first gives 2.50 (Module 1, Section 2.7).

Reduction over more than one dimension

A reduction method works on one dimension. For a two-dimensional array the elements of the outer array are themselves arrays, so the inner reduction is written inside the `with` clause:

```
int m [2][3] = '{ '{1, 2, 3}, '{4, 5, 6} };
int total;

total = m.sum() with (item.sum());         // item is one row; result is 21
```

This nested form is accepted by the major simulators, but support has been uneven historically. A pair of nested `foreach` loops is the portable alternative.

Reduction methods versus reduction operators

Do not confuse the array reduction **methods** with the reduction **operators** of Module 1.

	Reduction operator	Reduction method
Syntax	`^v`, `&v`, `|v` (unary, before the operand)	`arr.xor()`, `arr.and()`, `arr.or()`, `arr.sum()`, `arr.product()`
Works on	The **bits** of one packed vector	The **elements** of one unpacked array
Result	1 bit	A value as wide as one element
Example	`^8'b1011_0010` = 0 (four ones: even parity)	For `arr` = `'{8'hA0, 8'h0A}`, `arr.xor()` = `8'hAA`
Synthesizable	Yes	No

Interview trap: `bit [7:0] arr [4] = '{100, 100, 100, 100}; int s = arr.sum();`. What is `s`? It is 144, not 400. The sum is computed in 8 bits: 400 mod 256 = 144. Fix: `arr.sum() with (int'(item))`.

Interview trap: "Count the elements greater than 5 using `sum()`." The answer `arr.sum() with (item > 5)` is wrong because the expression is one bit wide. The correct answer is `arr.sum() with (int'(item > 5))`.

Common mistake: Computing an average with `arr.sum() / arr.size()` and expecting a fraction. Both operands are integers. Cast one to `real`.

Common mistake: Using `product()` on more than a few elements. The result overflows the element width very quickly and is silently truncated.

Verification application: (1) Checksums: `checksum = payload.xor();` gives the XOR of all payload bytes; `payload.sum() with (int'(item))` gives an additive checksum. (2) Constraints (Module 5): `constraint c { payload.sum() with (int'(item)) < 1000; }` limits the total of a random array, and the same width rule applies inside constraints. (3) End-of-test statistics: total bytes transferred, average latency. (4) Checking a one-hot array of grants: `assert (grant.sum() with (int'(item)) <= 1);`.

Interview questions

Q1. What are the array reduction methods? `sum()`, `product()`, `and()`, `or()` and `xor()`. Each combines all elements of an unpacked array into one value.

Q2. What is the width of the result of `sum()`? The width of the array element type, or of the `with` expression if one is given. It does not depend on the variable receiving the result.

Q3. `bit [7:0] a [3] = '{255, 1, 1}; $display("%0d", a.sum());` What is printed, and how do you get the true total? 1, because 257 is truncated to 8 bits. Use `a.sum() with (int'(item))` to get 257.

Q4. How do you compute the XOR checksum of a byte array? `chk = payload.xor();`

Q5. How do you add up only the elements at even indices?
`arr.sum() with (item.index % 2 == 0 ? item : 0)`

Q6. What is the difference between `^vec` and `arr.xor()`? `^vec` is the reduction operator: it XORs the bits of one packed vector and returns one bit. `arr.xor()` is the reduction method: it XORs the elements of an unpacked array and returns a value of the element width.

Q7. How do you find the total length of all packets in a queue of packet structs? `total = pkts.sum() with (item.len);`

Q8. Can reduction methods be used on an associative array? Yes. They combine the values of all existing entries.

Part 5: Array Initialisation, Assignment, Copying and Comparison

5.1 Array Initialisation

Summary of the forms

Array kind	In the declaration	Later, in procedural code
Packed	`logic [7:0] p = 8'hA5;`	`p = '0;` `p = {a, b};`
Fixed-size	`int a [4] = '{1, 2, 3, 4};`	`a = '{default: 0};`
Dynamic	`int d [] = '{1, 2, 3};` or `int d [] = new[3];`	`d = new[8];` `d = '{4, 5};`
Queue	`int q [$] = {1, 2, 3};`	`q = {};` `q.push_back(4);`
Associative	`int aa [string] = '{"a": 1, "b": 2};`	`aa["c"] = 3;`

Assignment pattern features

Form	Example	Result
Positional	`'{1, 2, 3, 4}`	One value per element, in index order from the left bound
Default	`'{default: 9}`	Every element is 9
Indexed	`'{0: 5, 3: 7, default: 0}`	Element 0 is 5, element 3 is 7, the rest 0
Replication	`'{4{8}}`	Four elements, each 8
Nested	`'{ '{1, 2}, '{3, 4} }`	One inner pattern per row of a two-dimensional array
Key and value	`'{"x": 1, "y": 2}`	Entries of an associative array

Rules:

- A positional pattern must supply **exactly** as many values as the fixed-size array has elements. For a dynamic array or queue, the number of values sets the size.
- Positional values cannot be mixed with `default:` or index keys in the same pattern.
- Elements that are never initialised keep the default of their type: X for four-state, 0 for two-state, `null` for class handles, `""` for strings.

Initialising with a loop

Patterns are convenient for small arrays. Larger arrays are filled procedurally:

```
logic [7:0] mem [256];
int         sq [];

initial begin
  foreach (mem[i]) mem[i] = i;                // address pattern: 00, 01, 02 ...
  sq = new[10];
  foreach (sq[i]) sq[i] = i * i;              // 0, 1, 4, 9 ...
end
```

Memories are often loaded from a file with `$readmemh("data.hex", mem);` (hexadecimal text) or `$readmemb` (binary text), which fill a fixed-size array of packed elements.

> **Common mistake:** Giving a declaration initialiser to a variable declared inside a task, function or `initial` block of a module, and expecting it to be applied each time the code runs. Such variables are static by default and are initialised once (Module 3). For arrays this shows up as "my array still has the values from the previous call". Assign in a separate statement, or declare the routine `automatic`.

5.2 Array Assignment and Copying

In plain English

Assigning one array to another with `=` copies **every element**. After the copy the two arrays are separate: changing one does not change the other. This is true for every array kind, and it is different from class handles, where `=` copies only a reference (Section 5.4).

Technical definition

An unpacked array can be assigned to another unpacked array when both have **equivalent element types** and **compatible shapes**:

Destination	Source	Rule
Fixed-size	Fixed-size	Same number of elements in every dimension. The index ranges may differ (`int a [0:3]` and `int b [7:4]` are compatible)
Dynamic or queue	Any array of the same element type	Always allowed. The destination is resized to match
Fixed-size	Dynamic or queue	Allowed by the compiler; a **run-time error** if the sizes differ
Associative	Associative with the same index type	Allowed. All entries are copied
Associative	Any non-associative array (and the reverse)	Not allowed

Elements are copied left to right by position: the leftmost element of the source goes to the leftmost element of the destination, whatever the index numbers are.

How it works internally

```
int a [3] = '{1, 2, 3};
int b [3];

b = a;                    a : [ 1 | 2 | 3 ]
                              |   |   |      every element is copied
                              v   v   v
                          b : [ 1 | 2 | 3 ]    b has its own storage

b[0] = 100;               a : [ 1   | 2 | 3 ]   unchanged
                          b : [ 100 | 2 | 3 ]
```

For dynamic arrays and queues the destination's old storage is released and new storage of the right size is allocated before the elements are copied.

Slices

A contiguous part of an array can be read or written as a unit:

```
int a [8];
int b [3];

b        = a[2:4];        // three elements out of a
a[5:7]   = b;             // three elements into a
a[0:1]   = a[6:7];        // copy within the same array
```

For fixed-size arrays the slice bounds must be constants (or use the indexed forms `a[i +: 3]` and `a[i -: 3]`, where the width is constant). For queues the bounds may be variables: `q[lo:hi]`.

Complete example

tb_array_assign.sv

```
module tb_array_assign;

  int fixed [3] = '{1, 2, 3};
  int other [3];
  int rng   [7:5];
  int dyn   [];
  int q     [$];

  initial begin
    other    = fixed;                 // fixed to fixed
    other[0] = 100;
    $display("1. fixed = %p   other = %p", fixed, other);

    rng = fixed;                      // different index range, same size
    $display("2. rng[7] = %0d   rng[5] = %0d", rng[7], rng[5]);

    dyn = fixed;                      // fixed to dynamic: dyn is resized to 3
    q   = dyn;                        // dynamic to queue
    q.push_back(4);
    dyn = q;                          // queue to dynamic: dyn is resized to 4
    $display("3. dyn = %p   size = %0d", dyn, dyn.size());

    fixed = dyn[1:3];                 // a 3-element slice fits the 3-element array
    $display("4. fixed = %p", fixed);

    other[0:1] = fixed[1:2];          // slice to slice
    $display("5. other = %p", other);

    // fixed = dyn;                   // dyn has 4 elements: this would be a run-time error
  end

endmodule
```

EXPECTED OUTPUT

```
1. fixed = '{1, 2, 3}   other = '{100, 2, 3}
2. rng[7] = 1   rng[5] = 3
3. dyn = '{1, 2, 3, 4}   size = 4
4. fixed = '{2, 3, 4}
5. other = '{3, 4, 3}
```

Line by line

Line	Explanation
1	`other = fixed` copied three values. Changing `other[0]` afterwards leaves `fixed` alone.
2	`rng` is declared `[7:5]`. The copy is positional: the leftmost source element `fixed[0]` goes to the leftmost destination element `rng[7]`.
3	A dynamic array and a queue take whatever size the source has.
4	`dyn[1:3]` is `{2, 3, 4}`.
5	`fixed[1:2]` is `{3, 4}`; it replaces elements 0 and 1 of `other`, whose element 2 still holds 3.

Copying when arrays are passed to tasks and functions

Passing an array as an ordinary (`input`) argument **copies the whole array** on every call. For a large array this is slow. Declaring the argument `ref` passes a reference instead, and `const ref` does so while preventing the routine from modifying the caller's array. Module 3 explains argument passing in detail.

```
function automatic int total_by_value(int arr []);           // copies arr on every call
function automatic int total_by_ref(const ref int arr []);    // no copy, read-only
```

Interview trap: "Does `b = a` for two arrays make `b` refer to the same storage as `a`?" No. Arrays have value semantics: the elements are copied and the arrays are independent. Only **class handles** have reference semantics.

Common mistake: Assigning a dynamic array to a fixed-size array without checking the size. It compiles, and then fails at run time when the sizes differ.

Common mistake: Assuming the copy matches index numbers. `int a [0:3]` assigned to `int b [3:0]` puts `a[0]` into `b[3]`, because both are the leftmost element.

Interview questions

Q1. How do you copy one array to another? With a simple assignment, `b = a;`. All elements are copied. A dynamic array or queue destination is resized automatically.

Q2. When are two fixed-size arrays assignment-compatible? When their element types are equivalent and they have the same number of dimensions with the same number of elements in each. The index ranges need not be identical.

Q3. What happens when a 5-element dynamic array is assigned to a 4-element fixed-size array? A run-time error. The sizes must match because a fixed-size array cannot be resized.

Q4. After `q2 = q1;` and `q2.push_back(9);`, does `q1` change? No. `q2` is an independent copy.

Q5. How do you copy only part of an array? With a slice: `b = a[2:5];` for a fixed range, or `q2 = q1[lo:hi];` for a queue.

5.3 Array Comparison

Rules

- Two unpacked arrays of compatible type can be compared as a whole with `==` and `!=`. The comparison is element by element, in position order, and gives a single-bit result.
- Dynamic arrays and queues of different sizes are simply not equal.
- Slices can be compared: `a[0:3] == b[4:7]`.
- With four-state elements, `==` follows the rules of Module 1, Section 4.5: an X or Z in a compared element can make the whole result X. Use `===` and `!==` in checkers so that the result is always 0 or 1.
- Relational operators (`<`, `>`) are **not** defined for unpacked arrays.
- When the elements are class handles, the handles are compared, not the objects they point to (Section 5.4).

Complete example

tb_array_compare.sv

```
module tb_array_compare;

  logic [7:0] exp_data [4] = '{8'h11, 8'h22, 8'h33, 8'h44};
  logic [7:0] act_data [4] = '{8'h11, 8'h22, 8'h3F, 8'h44};
  logic [7:0] x_data   [4] = '{8'h11, 8'h22, 8'hxx, 8'h44};
  int         q1 [$] = {1, 2, 3};
  int         q2 [$] = {1, 2, 3, 4};

  initial begin
    if (act_data == exp_data) $display("1. arrays match");
    else                      $display("1. arrays differ");

    foreach (exp_data[i])
      if (act_data[i] !== exp_data[i])
        $display("2.   mismatch at [%0d]: actual=%h expected=%h",
                 i, act_data[i], exp_data[i]);

    $display("3. first two elements equal : %b", act_data[0:1] == exp_data[0:1]);

    if (x_data != exp_data)  $display("4. using != : mismatch reported");
    else                     $display("4. using != : no mismatch reported");
    if (x_data !== exp_data) $display("5. using !== : mismatch reported");

    $display("6. different sizes equal? %b", q1 == q2);
    q2.delete(3);
    $display("7. after delete(3) equal? %b", q1 == q2);
  end

endmodule
```

EXPECTED OUTPUT

```
1. arrays differ
2.   mismatch at [2]: actual=3f expected=33
3. first two elements equal : 1
4. using != : no mismatch reported
5. using !== : mismatch reported
6. different sizes equal? 0
7. after delete(3) equal? 1
```

Line by line

Line	Explanation
1	One comparison for the whole array: it tells you *that* they differ, not *where*.
2	The usual follow-up in a scoreboard: a loop that reports each differing element.
3	Slices are compared like whole arrays.
4	Element 2 of `x_data` is X. The comparison of that element is X, so the whole `!=` is X, which the `if` treats as false. The mismatch is missed.
5	`!==` compares X literally and reports the mismatch. This is the array form of the scoreboard trap from Module 1.
6	Arrays of different sizes are not equal.
7	`delete(3)` removes the element at index 3 of the queue `q2`. Both queues now hold `{1, 2, 3}` and compare equal.

> **Common mistake:** Comparing arrays of DUT data with `==` or `!=` in a scoreboard. One X element makes the result X and the failure disappears. Use `!==`.

> **Verification application:** A monitor collects the bytes of a packet into a queue; the scoreboard compares that queue with the expected payload in a single statement, `if (act.payload !== exp.payload)`, and only loops over the elements to print details when the comparison fails.

Interview questions

Q1. How do you compare two arrays? With `==` or `!=` (or `===` / `!==` for four-state data) on the whole arrays. The element types must be equivalent. The result is one bit.

Q2. What is the result of comparing two queues of different sizes? They are not equal: `==` gives 0.

Q3. Why prefer `!==` over `!=` when comparing arrays of `logic`? An X or Z in any element makes `!=` return X, which an `if` treats as false, so a mismatch is not reported. `!==` always returns 0 or 1.

Q4. Which arrays support `delete(index)`? Queues and associative arrays. A dynamic array supports only `delete()` without an argument, which removes all elements.

5.4 Shallow Copy and Deep Copy

In plain English

When the elements of an array are plain values (numbers, structs, strings), copying the array copies the values, and that is the end of it.

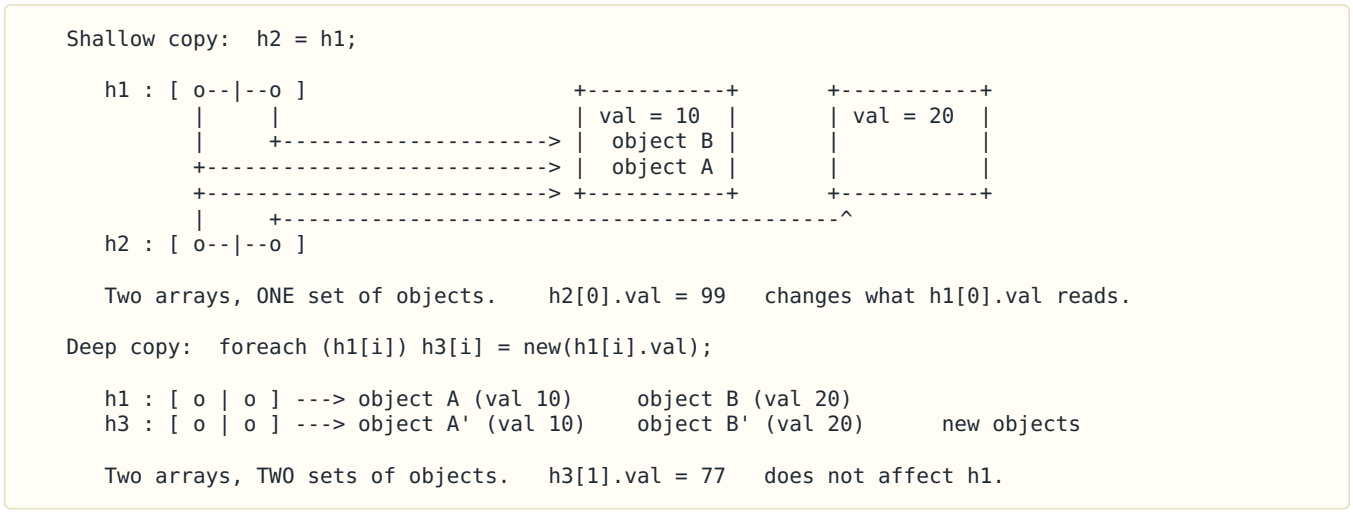
When the elements are **class handles**, each element is not the object itself but a reference to an object that lives elsewhere. Copying the array copies the references. Both arrays then refer to the **same objects**. This is a **shallow copy**. If you need a second, independent set of objects, you must create new objects and copy the contents of each one. This is a **deep copy**.

Classes and handles are the subject of Module 4. The small class below is enough to show the effect on arrays: `new(10)` creates an object holding 10 and returns a handle to it.

Technical definition

Term	What is copied	Are the two arrays independent?
Array copy of values	Every element value	Yes
Shallow copy of an array of handles	The handles only	The arrays are separate, but they point to the same objects
Deep copy of an array of handles	A new object is created for each element and the object's contents are copied into it	Yes, completely

How it works internally



Complete example

tb_array_copy.sv

```
module tb_array_copy;

  class item_c;
    int val;
    function new(int v);
      val = v;
    endfunction
  endclass

  int    a [3] = '{1, 2, 3};
  int    b [3];
  item_c h1 [2];
  item_c h2 [2];
  item_c h3 [2];

  initial begin
    // arrays of values: a real copy
    b    = a;
    b[0] = 100;
    $display("1. values : a[0] = %0d   b[0] = %0d", a[0], b[0]);

    // arrays of class handles
    h1[0] = new(10);
    h1[1] = new(20);

    h2 = h1;                                  // shallow copy: only the handles are copied
    h2[0].val = 99;
    $display("2. shallow: h1[0].val = %0d   h2[0].val = %0d", h1[0].val, h2[0].val);

    foreach (h1[i]) h3[i] = new(h1[i].val);   // deep copy: a new object per element
    h3[1].val = 77;
    $display("3. deep   : h1[1].val = %0d   h3[1].val = %0d", h1[1].val, h3[1].val);

    $display("4. h1 == h2 : %b    h1 == h3 : %b", h1 == h2, h1 == h3);
  end

endmodule
```

EXPECTED OUTPUT

```
1. values : a[0] = 1   b[0] = 100
2. shallow: h1[0].val = 99   h2[0].val = 99
3. deep   : h1[1].val = 20   h3[1].val = 77
4. h1 == h2 : 1    h1 == h3 : 0
```

Line by line

Line	Explanation
1	`int` elements: `b` is a true copy, so the change to `b[0]` is not seen in `a`.
2	`h2 = h1` copies two handles. `h2[0]` and `h1[0]` refer to the same object, so writing 99 through `h2[0]` is visible through `h1[0]`.
3	The loop creates a new object for each element and initialises it with the value from the original. `h3[1]` is a different object from `h1[1]`, so changing it has no effect on `h1`.
4	Comparing arrays of handles compares the handles. `h1` and `h2` hold identical handles: equal. `h3` holds handles to different objects, so it is not equal to `h1`, even though at the moment of copying the object contents were the same.

Comparison table

Property	Array of values (`int`, struct, `bit [7:0]`)	Array of class handles
What an element holds	The data	A reference to an object
`b = a;`	Copies the data. Fully independent	Copies the references. Objects are shared (shallow)
How to get an independent copy	`b = a;` is enough	Loop: create a new object per element and copy its fields (deep)
`a == b`	Compares the data	Compares the references
Default element value	0 or X	`null`
Memory after `delete()`	Elements are released	Handles are released; an object is freed only when no handle refers to it any more

Interview trap: "I pushed the same transaction object into a queue ten times inside a loop, changing its fields each time. Why do all ten queue entries show the last values?" Because the queue holds ten copies of the **same handle**. Only one object exists. A new object must be created (or the object copied) for each entry. This bug appears in almost every beginner's generator and is covered again in Modules 4 and 9.

Common mistake: Believing that `new[n]` on a dynamic array of handles creates `n` objects. It creates `n` handles, all `null`. Each object must still be constructed: `foreach (arr[i]) arr[i] = new();`.

Verification application: A scoreboard that stores received transactions in a queue must store copies if the sender keeps reusing the same object. Transaction classes therefore provide a `copy()` method (as `alu_txn` does) and components write `exp_q.push_back(txn.copy());`.

Interview questions

Q1. What is the difference between a shallow copy and a deep copy? A shallow copy duplicates the handles, so the original and the copy refer to the same objects. A deep copy creates a new object for each original object and copies its contents, so the two are fully independent.

Q2. Is `b = a` for two arrays of `int` shallow or deep? The distinction does not apply: the elements are values, and the assignment copies them. The arrays are independent.

Q3. What does `arr = new[5];` create when `arr` is a dynamic array of class handles? Five `null` handles. No objects are created until `new()` is called for each element.

Q4. How do you deep-copy a queue of transactions? Loop over the source and push a copy of each object: `foreach (src[i]) dst.push_back(src[i].copy());`.

Part 6: Loops and Array Iteration

6.1 The for Loop

In plain English

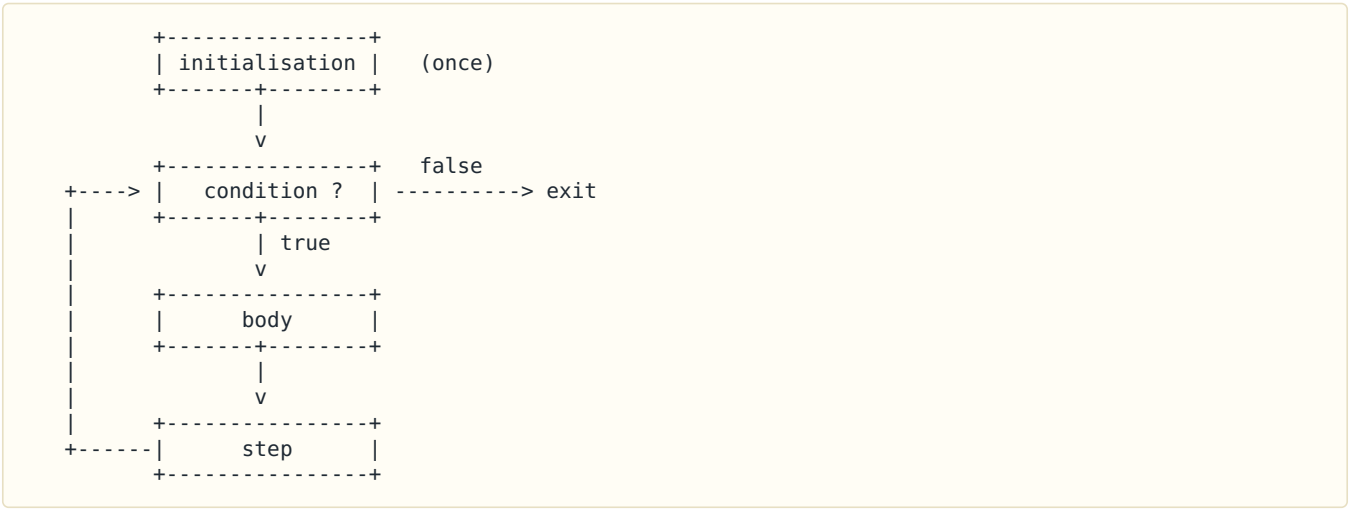
A `for` loop repeats a block of code while counting. You state where the counter starts, the condition under which the loop continues, and how the counter changes after each pass.

Syntax

```
for (initialisation; condition; step) statement

for (int i = 0; i < 8; i++) begin
  ...
end
```

Execution order: the initialisation runs once; then the condition is tested **before** every pass; if it is true the body runs, then the step, then the condition is tested again.



SystemVerilog additions compared with Verilog:

- The loop variable can be declared inside the loop (`for (int i = 0; ...`). It is then local to the loop, which prevents accidental sharing between loops running in parallel threads.
- Several variables can be initialised and stepped, separated by commas.
- `i++`, `i--`, `i += 2` are available.

Complete example

tb_loop_for.sv

```
module tb_loop_for;

  int arr [5] = '{3, 8, 1, 6, 4};
  int total;
  int tmp;

  initial begin
    total = 0;
    for (int i = 0; i < $size(arr); i++) total += arr[i];
    $display("1. total = %0d", total);

    $write("2. backwards            :");
    for (int i = $high(arr); i >= $low(arr); i--) $write(" %0d", arr[i]);
    $display("");

    $write("3. every second element :");
    for (int i = 0; i < $size(arr); i += 2) $write(" %0d", arr[i]);
    $display("");

    // two loop variables: reverse the array in place
    for (int lo = 0, hi = $size(arr) - 1; lo < hi; lo++, hi--) begin
      tmp     = arr[lo];
      arr[lo] = arr[hi];
      arr[hi] = tmp;
    end
    $display("4. reversed in place    : %p", arr);
  end

endmodule
```

EXPECTED OUTPUT

```
1. total = 22
2. backwards            : 4 6 1 8 3
3. every second element : 3 1 4
4. reversed in place    : '{4, 6, 1, 8, 3}
```

Line	Explanation
1	The classic counting loop. `$size(arr)` keeps the bound correct if the array is resized in the source. 3 + 8 + 1 + 6 + 4 = 22.
2	Counting down from the highest index. The loop variable is a signed `int`, so `i >= 0` becomes false when `i` reaches -1.
3	A step other than 1: indices 0, 2, 4.
4	Two variables move toward each other and the elements are swapped. The pairs are (0,4) and (1,3); when `lo` and `hi` meet at 2 the loop ends.

Interview trap: `for (int unsigned i = n - 1; i >= 0; i--)` never terminates. An unsigned variable is always greater than or equal to 0; after 0 it wraps to 4,294,967,295. Use a signed `int` for a loop that counts down to zero.

Common mistake: `for (int i = 0; i <= arr.size(); i++)`. The `<=` makes one pass too many and reads one element past the end. The condition is `i < arr.size()`.

6.2 The foreach Loop

In plain English

"For each element of this array, do the following." You do not write the bounds or the step. The loop works them out from the array, so it is always right, whatever the array's size or index range.

Technical definition

`foreach` iterates over the indices of an array. The loop variable named in the brackets is declared implicitly; it is local to the loop, has the type of the array's index, and cannot be assigned inside the loop. Iteration proceeds from the left bound of the dimension to its right bound; for dynamic arrays and queues from 0 to `size()-1`; for associative arrays in key order.

Syntax

```
foreach (arr[i])       statement      // one dimension
foreach (m[i, j])      statement      // two dimensions: i is the outer index
foreach (m[i])         statement      // only the first dimension
foreach (m[, j])       statement      // only the second dimension (first one skipped)
```

What foreach can iterate over

Operand	The loop variable takes
Fixed-size array `int a [3:1]`	3, 2, 1 (left bound to right bound)
Dynamic array, queue	0 to `size()-1`
Associative array	Each existing key, in sorted order
Packed vector `logic [3:0] v`	3, 2, 1, 0
`string`	0 to `len()-1` (each character)

Complete example

tb_loop_foreach.sv

```
module tb_loop_foreach;

  int         arr [5]  = '{3, 8, 1, 6, 4};
  int         rev [3:1] = '{10, 20, 30};
  int         q [$]    = {7, 8};
  int         aa [string] = '{"b": 2, "a": 1};
  logic [3:0] nib      = 4'b1010;
  string      s        = "SV";
  int         max_val, max_idx;

  initial begin
    max_val = arr[0];
    max_idx = 0;
    foreach (arr[i])
      if (arr[i] > max_val) begin
        max_val = arr[i];
        max_idx = i;
      end
    $display("1. largest is %0d at index %0d", max_val, max_idx);

    $write("2. fixed [3:1] :");
    foreach (rev[i]) $write(" rev[%0d]=%0d", i, rev[i]);
    $display("");

    $write("3. queue       :");
    foreach (q[i]) $write(" q[%0d]=%0d", i, q[i]);
    $display("");

    $write("4. associative :");
    foreach (aa[k]) $write(" aa[%s]=%0d", k, aa[k]);
    $display("");

    $write("5. packed bits :");
    foreach (nib[i]) $write(" nib[%0d]=%b", i, nib[i]);
    $display("");

    $write("6. string      :");
    foreach (s[i]) $write(" s[%0d]=%c", i, s[i]);
    $display("");
  end

endmodule
```

EXPECTED OUTPUT

```
1. largest is 8 at index 1
2. fixed [3:1] : rev[3]=10 rev[2]=20 rev[1]=30
3. queue       : q[0]=7 q[1]=8
4. associative : aa[a]=1 aa[b]=2
5. packed bits : nib[3]=1 nib[2]=0 nib[1]=1 nib[0]=0
6. string      : s[0]=S s[1]=V
```

Line	Explanation
1	A typical search loop: remember the best value and its index.
2	The array is declared `[3:1]`. The pattern `'{10, 20, 30}` fills it from the left bound, so `rev[3]` is 10. `foreach` also starts at the left bound: 3, 2, 1.
3	A queue is visited from 0 upward.
4	The keys come out sorted (`"a"` before `"b"`), not in the order written in the initialiser. `k` is a `string`.
5	A packed vector is visited from its left bound (bit 3) to its right bound (bit 0).

Line	Explanation
6	Each element of a string is a character (a `byte`); `%c` prints it as a character.

for versus foreach

Property	`for`	`foreach`
Bounds	Written by you	Taken from the array
Step	Any	Always one element, in declaration order
Direction	Either	Left bound to right bound only
Loop variable	Declared by you; can be modified	Implicit; read-only
Partial ranges, skipping elements	Yes	No (use `if` / `continue` inside)
Associative arrays	Needs `first()` / `next()`	Direct
Multidimensional	Nested loops	One statement: `foreach (m[i, j])`
Risk of off-by-one errors	Yes	None
Usable in constraints	No	Yes (Module 5)

Use `foreach` to visit every element. Use `for` when you need a different direction, step or range.

> **Interview trap:** "Can you assign to an element inside `foreach`?" Yes: `foreach (arr[i]) arr[i] = 0;` is fine. What you cannot assign is the loop variable `i`.

> **Common mistake:** Declaring the `foreach` index variable yourself. `int i; foreach (arr[i]) ...` creates a **new** implicit `i` inside the loop; the outer `i` is untouched and still has its old value afterwards.

6.3 The repeat Loop

Definition and syntax

`repeat (n) statement` executes the statement exactly `n` times. There is no loop variable and no condition. The count `n` is evaluated **once**, before the first pass; changing the variables in the count expression inside the body does not change the number of passes. If `n` is zero or negative the body does not run; if it is X or Z it is treated as zero.

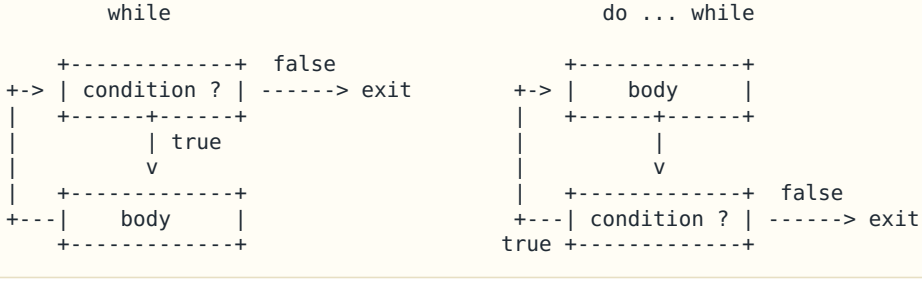
```
repeat (5) @(posedge clk);            // wait for five rising clock edges
repeat (num_txn) begin                // generate num_txn transactions
  ...
end
```

`repeat` is the clearest loop when the number of passes is known and the pass number is not needed in the body. Its most common use in testbenches is waiting a number of clock cycles.

6.4 The while and do-while Loops

Definition

Loop	Syntax	Condition tested	Minimum passes
`while`	`while (cond) statement`	**Before** each pass	0
`do` … `while`	`do statement while (cond);`	**After** each pass	1



Both are used when the number of passes is not known in advance: "keep going until something happens". A condition that is X or Z is treated as false.

Complete example: repeat, while, do-while, break and continue

tb_loop_while.sv

```
module tb_loop_while;

  int q [$] = {5, 12, 7, 20, 3};
  int count, x, idx, n;

  initial begin
    // repeat: a fixed number of passes
    count = 0;
    repeat (4) count += 2;
    $display("1. repeat (4)          : count = %0d", count);

    // the repeat count is evaluated once
    count = 3;
    repeat (count) count++;
    $display("2. repeat (count)      : count = %0d", count);

    // while: search for the first element greater than 10
    idx = 0;
    while (idx < q.size() && q[idx] <= 10) idx++;
    $display("3. first element > 10  : q[%0d] = %0d", idx, q[idx]);

    // while may run zero times, do-while runs at least once
    x = 100;
    count = 0;
    while (x < 10) count++;
    $display("4. while (false)       : %0d passes", count);
    count = 0;
    do count++; while (x < 10);
    $display("5. do-while (false)    : %0d pass", count);

    // while: drain a queue
    $write("6. drained             :");
    while (q.size() > 0) begin
      x = q.pop_front();
      $write(" %0d", x);
    end
    $display("");
```

```
    // continue and break
    $write("7. continue and break  :");
    for (int i = 0; i < 10; i++) begin
      if (i % 2 == 1) continue;        // skip odd numbers
      if (i > 6)      break;           // stop the loop
      $write(" %0d", i);
    end
    $display("");

    // forever with break
    n = 0;
    forever begin
      n++;
      if (n == 5) break;
    end
    $display("8. forever with break  : n = %0d", n);
  end

endmodule
```

EXPECTED OUTPUT

```
1. repeat (4)          : count = 8
2. repeat (count)      : count = 6
3. first element > 10  : q[1] = 12
4. while (false)       : 0 passes
5. do-while (false)    : 1 pass
6. drained             : 5 12 7 20 3
7. continue and break  : 0 2 4 6
8. forever with break  : n = 5
```

Line	Explanation
1	Four passes, each adding 2.
2	The count is read once (3). The body changes `count`, but the loop still runs exactly three times: 3 → 6.
3	The condition has two parts. `idx < q.size()` is checked first; because `&&` short-circuits, `q[idx]` is never read with an invalid index. The loop stops at index 1 (the value 12).
4	The condition is false at the start, so the `while` body never runs.
5	The `do` ... `while` body runs once before the condition is tested.
6	The standard way to empty a queue while processing every element.
7	`continue` jumps to the next pass (to the step `i++`); `break` leaves the loop. Even numbers 0, 2, 4, 6 are printed; at `i` = 8 the loop is left.
8	`forever` has no condition. In testbench components it normally contains a timing control (`@(posedge clk)`, a mailbox `get`) and runs for the whole simulation. Here it is ended by `break`. A `forever` loop with neither a timing control nor a `break` hangs the simulator at one time step.

break, continue and return

Statement	Effect
`break`	Leaves the innermost enclosing loop immediately
`continue`	Skips the rest of the current pass and goes to the next one
`return`	Leaves the whole task or function (and therefore every loop in it)

All three are SystemVerilog additions; Verilog needed `disable` of a named block.

Interview trap: "What is the difference between `while` and `do ... while`?" The `while` loop tests the condition first and may execute zero times. The `do ... while` loop executes the body first and tests afterwards, so it executes at least once. The associative array traversal `if (aa.first(k)) do ... while (aa.next(k));` relies on this: the body must run for the first key before `next` is called.

Common mistake: A `while` or `forever` loop in a testbench with no statement that lets simulation time advance and no exit condition that can become true. The simulation appears to hang at a fixed time. Every such loop needs either a timing control or a guaranteed exit.

6.5 Nested Loops

Definition

A nested loop is a loop inside the body of another loop. For each pass of the outer loop, the inner loop runs completely. Nested loops are used for multidimensional arrays, for comparing every element with every other element, and for sorting.

`break` and `continue` act on the **innermost** loop only. To leave both loops, use a flag tested by the outer loop, or put the loops in a function and `return`.

Complete example

tb_loop_nested.sv

```
module tb_loop_nested;

  int arr [5] = '{5, 2, 9, 1, 7};
  int tmp, swaps;
  bit found;

  initial begin
    // 1. triangle: the inner bound depends on the outer variable
    for (int i = 1; i <= 3; i++) begin
      $write("1. row %0d:", i);
      for (int j = 1; j <= i; j++) $write(" %0d", i * j);
      $display("");
    end

    // 2. bubble sort
    swaps = 0;
    for (int i = 0; i < $size(arr) - 1; i++)
      for (int j = 0; j < $size(arr) - 1 - i; j++)
        if (arr[j] > arr[j + 1]) begin
          tmp        = arr[j];
          arr[j]     = arr[j + 1];
          arr[j + 1] = tmp;
          swaps++;
        end
    $display("2. sorted = %p after %0d swaps", arr, swaps);

    // 3. find the first pair of elements that adds up to 10
    found = 0;
    for (int i = 0; i < $size(arr) && !found; i++)
      for (int j = i + 1; j < $size(arr); j++)
        if (arr[i] + arr[j] == 10) begin
          $display("3. %0d + %0d = 10", arr[i], arr[j]);
          found = 1;
          break;                       // leaves the inner loop only
        end
  end

endmodule
```

EXPECTED OUTPUT

```
1. row 1: 1
1. row 2: 2 4
1. row 3: 3 6 9
2. sorted = '{1, 2, 5, 7, 9} after 5 swaps
3. 1 + 9 = 10
```

Part	Explanation
1	The inner loop runs `i` times for outer value `i`, producing a triangle.
2	Bubble sort. Pass 0 moves the largest value to the end: `{2, 5, 1, 7, 9}` after three swaps. Pass 1: `{2, 1, 5, 7, 9}` (one swap). Pass 2: `{1, 2, 5, 7, 9}` (one swap). Pass 3: no swap. Five swaps in total. In real code you would write `arr.sort();` but interviewers ask for the loop version.
3	On the sorted array, the first pair found is `arr[0]` = 1 with `arr[4]` = 9. `break` leaves the inner loop; the `!found` term in the outer condition then ends the outer loop.

6.6 Loop Summary and Iterating Safely

All loops compared

Loop	Use when	Number of passes	Loop variable
`for`	Counting with a known start, end and step; partial or reverse traversal	Determined by the condition	Explicit
`foreach`	Visiting every element of an array	One per element	Implicit, read-only
`repeat (n)`	Doing something exactly `n` times	`n`, evaluated once	None
`while`	Continuing as long as a condition holds; may not run at all	0 or more	None (your own)
`do ... while`	As `while`, but the body must run at least once	1 or more	None (your own)
`forever`	Processes that run for the whole simulation	Unlimited, until `break`, `return` or `disable`	None

All loop statements are usable in testbenches. In synthesizable RTL, `for`, `foreach` and `repeat` with constant bounds are accepted, because the tool unrolls them into parallel hardware; a loop does not become a sequential circuit.

Changing an array while looping over it

Removing elements from a queue while walking forward through it skips elements, because every deletion shifts the following elements down by one.

tb_loop_delete.sv

```
module tb_loop_delete;

  int q1 [$] = {1, 2, 4, 5, 6};
  int q2 [$] = {1, 2, 4, 5, 6};
  int q3 [$] = {1, 2, 4, 5, 6};

  initial begin
    // WRONG: forward loop, deleting even values
    for (int i = 0; i < q1.size(); i++)
      if (q1[i] % 2 == 0) q1.delete(i);
    $display("1. forward delete  : %p", q1);

    // RIGHT: walk backwards, so the shifted elements have already been visited
    for (int i = q2.size() - 1; i >= 0; i--)
      if (q2[i] % 2 == 0) q2.delete(i);
    $display("2. backward delete : %p", q2);

    // RIGHT: build the result with a locator method
    q3 = q3.find with (item % 2 != 0);
    $display("3. find            : %p", q3);
  end

endmodule
```

EXPECTED OUTPUT

```
1. forward delete  : '{1, 4, 5}
2. backward delete : '{1, 5}
3. find            : '{1, 5}
```

Trace of the wrong loop: at `i` = 1 the value 2 is deleted and the queue becomes `{1, 4, 5, 6}`. The value 4 has moved to index 1, but `i` advances to 2, so 4 is never examined. At `i` = 3 the value 6 is deleted. The even value 4 survives.

The same hazard exists with `foreach` (whose behaviour is not defined when the array size changes during the loop) and with associative arrays.

> **Common mistake:** Deleting from a queue inside a forward loop. Loop backwards, or use `find` to build the filtered queue, or (when using `while`) advance the index only when nothing was deleted.

Interview questions

Q1. Which loops does SystemVerilog provide? `for`, `foreach`, `repeat`, `while`, `do ... while` and `forever`, with `break` and `continue` for control.

Q2. What is the difference between `for` and `foreach`? `for` has explicit initialisation, condition and step, so it can cover any range in any direction. `foreach` visits every index of an array automatically, with an implicit read-only loop variable, and works directly on associative arrays and multiple dimensions.

Q3. What is the difference between `while` and `do ... while`? `while` tests the condition before the body and may run zero times. `do ... while` tests after the body and always runs at least once.

Q4. In `repeat (n)`, what happens if `n` changes inside the loop? Nothing. The count is evaluated once, before the first pass.

Q5. What do `break` and `continue` do in nested loops? They affect only the innermost loop that contains them.

Q6. In what order does `foreach` traverse `int a [5:1]`? From the left bound to the right bound: 5, 4, 3, 2, 1.

Q7. How do you loop over a two-dimensional array? `foreach (m[i, j])`, or two nested `for` loops using `$size(m, 1)` and `$size(m, 2)`.

Q8. Are loops synthesizable? `for`, `foreach` and `repeat` loops with bounds that are constant at compile time are synthesizable and are unrolled into replicated logic. Loops whose number of passes depends on run-time data, and `forever` loops without a clock event, are not.

Q9. How do you safely delete elements from a queue while iterating over it? Iterate from the last index down to 0, or replace the queue with the result of `find with (...)` that keeps the wanted elements.

Q10. Why does a `forever` loop without a delay hang the simulation? The simulator executes the loop endlessly within the current time step and never gets the opportunity to advance simulation time or run other processes.

Part 7: Module 2 Revision and Practice

7.1 Revision Summary

The array kinds on one page

Kind	Declaration	Size	Allocate	Count	Remove	Literal
Packed	`logic [7:0] p;`	Fixed	n/a	`$size(p)`, `$bits(p)`	n/a	`8'hA5`, `{a, b}`
Fixed-size	`int a [8];`	Fixed	n/a	`$size(a)`	n/a	`'{1, 2, ...}`
Dynamic	`int d [];`	Run time	`d = new[n];` `d = new[n](d);`	`d.size()`	`d.delete()`	`'{1, 2, 3}`
Queue	`int q [$];`	Automatic	Automatic	`q.size()`	`q.delete(i)`, `q.delete()`, `pop_*`	`{1, 2, 3}`
Associative	`int aa [key_t];`	Automatic	On first write	`aa.num()`, `aa.size()`	`aa.delete(k)`, `aa.delete()`	`'{k1: v1, k2: v2}`

Methods by array kind

Kind	Methods specific to it
Dynamic	`new[]`, `size()`, `delete()`
Queue	`size()`, `push_back()`, `push_front()`, `pop_back()`, `pop_front()`, `insert(i, v)`, `delete(i)`, `delete()`
Associative	`num()`, `size()`, `exists(k)`, `delete(k)`, `delete()`, `first(k)`, `last(k)`, `next(k)`, `prev(k)`
All unpacked	Locator: `find`, `find_index`, `find_first`, `find_first_index`, `find_last`, `find_last_index`, `min`, `max`, `unique`, `unique_index`
All unpacked except associative	Ordering: `sort`, `rsort`, `reverse`, `shuffle`
All unpacked	Reduction: `sum`, `product`, `and`, `or`, `xor`
Every array	Query functions: `$size`, `$dimensions`, `$unpacked_dimensions`, `$left`, `$right`, `$low`, `$high`, `$increment`, `$bits`

Rules to remember

Fixed-size arrays

- `int a [5]` means `[0:4]`. The size must be a constant.
- Use `$size(a)`; the `size()` method is not defined for fixed-size arrays.
- Out-of-range read returns X (four-state) or 0 (two-state); out-of-range write is ignored.

Query functions

- Dimensions are numbered unpacked first (left to right), then packed (left to right).
- `$left` / `$right` follow the declaration; `$low` / `$high` follow the numbers.
- `$dimensions(int a [4])` is 2, because `int` is itself a packed vector.

Dynamic arrays

- Size 0 until `new[n]`. `new[n]` discards the contents; `new[n](d)` keeps them.
- Never grows automatically. `d[d.size()] = x` is ignored.
- `d = other;` resizes and copies.

Queues

- `q[0]` is the first element, `q[$]` the last. `q[$+1] = x` appends.
- `push_back` + `pop_front` is a FIFO. `push_back` + `pop_back` is a stack.
- `pop_*` on an empty queue returns the default value with a warning. Check `size()` first.
- `delete(i)` removes one element; `delete()` removes all.

Associative arrays

- Entries exist only after being written. Reading a missing key returns the default value and creates nothing.
- Test with `exists()`, never by comparing the value with 0.
- `first`, `last`, `next`, `prev` return a status (1/0) and deliver the key through their argument.
- Traversal is always in sorted key order; signed index types put negative keys first.
- No `sort`, `rsort`, `reverse`, `shuffle`.

Array methods

- Every locator method returns a **queue**, possibly empty. `find...` methods require a `with` clause.
- Index-returning methods return `int` (or the index type for an associative array).
- Ordering methods work in place and return nothing. `reverse` and `shuffle` take no `with`.
- A reduction result has the width of the element, or of the `with` expression. Use `int'(item)` to avoid overflow and `int'(condition)` to count.

Copying and comparing

- `b = a` copies all elements; the arrays are independent afterwards.
- Arrays of class handles copy the handles only (shallow). A deep copy needs a loop that creates new objects.
- `==` / `!=` compare whole arrays; use `===` / `!==` for four-state data.
- Fixed = dynamic compiles but fails at run time if the sizes differ.

Loops

- `foreach` goes from the left bound to the right bound; its variable is implicit and read-only; two dimensions are written `foreach (m[i, j])`.
- `while` may run zero times; `do` ... `while` runs at least once.
- `repeat (n)` evaluates `n` once.
- `break` and `continue` act on the innermost loop.
- Do not delete from a queue while looping forward over it.

Choosing an array

If the data is...	Use
A bus or bit field treated as one number	Packed array
A fixed number of items	Fixed-size array
A block whose size is decided at run time	Dynamic array
A list that grows and shrinks, processed in order	Queue
Sparse, or indexed by address, ID or string	Associative array

7.2 Interview Questions and Answers

The questions inside Parts 1 to 6 cover each topic directly. The following are additional and are grouped by difficulty.

Basic level

Q1. Name the unpacked array types in SystemVerilog. Fixed-size (static) arrays, dynamic arrays, queues and associative arrays.

Q2. How do you declare each of them for elements of type `int`? `int a [10];`, `int d [];`, `int q [$];`, `int aa [string];` (or another index type).

Q3. What is the difference between `new[]` and `new()`? `new[n]` allocates the elements of a dynamic array. `new()` constructs a class object.

Q4. What is the size of a dynamic array before `new[]` is called? Zero.

Q5. What does `q.pop_front()` do? It removes the first element of the queue and returns it.

Q6. How do you add an element at index 2 of a queue? `q.insert(2, value);`

Q7. How do you check whether a key is present in an associative array? `aa.exists(key)`

Q8. How do you get the number of entries of an associative array? `aa.num()` or `aa.size()`.

Q9. What is the default value of an element of `logic [7:0] mem [16]`? Of `int a [16]`? X for every bit of the `logic` elements; 0 for the `int` elements.

Q10. Which loop would you use to visit every element of an array? `foreach (arr[i])`.

Q11. What does `arr.sum()` return for `int arr [3] = '{1, 2, 3}`? 6.

Q12. What does `sort()` do? It sorts the array in place in ascending order.

Q13. How do you empty a queue? `q.delete();` or `q = {};`

Q14. What does `$size` return for `int m [3][4]`? 3. With a second argument, `$size(m, 2)` returns 4.

Q15. How do you print a whole array? With the `%p` format specifier, or with a `foreach` loop.

Intermediate level

Q16. What is the difference between a dynamic array and an associative array? A dynamic array has contiguous integer indices from 0, must be allocated with `new[]`, and stores every element. An associative array has an index of any declared type, allocates entries one by one when written, and stores only the entries that exist. The dynamic array is faster to index; the associative array saves memory for sparse data.

Q17. Both a queue and a dynamic array can change size. When is a queue the better choice? When single elements are added or removed frequently, especially at the ends. A queue does this in constant time, whereas a dynamic array must be reallocated and copied for every change of size.

Q18. `int d [] = new[5]; d[5] = 10;` **What happens?** Index 5 is outside the range 0 to 4. The write is ignored (with a warning) and the array keeps its size. A dynamic array never grows on assignment.

Q19. How do you add an element to the end of a dynamic array? `d = new[d.size() + 1](d); d[d.size() - 1] = value;`. If this is needed often, use a queue.

Q20. How do you iterate over an associative array in reverse key order?

```
if (aa.last(k))
  do $display("%0d", aa[k]);
  while (aa.prev(k));
```

Q21. What is the difference between `find_first_index()` **and** `find_last_index()`**?** Both return a queue with at most one index: the lowest and the highest index, respectively, at which the `with` condition is true.

Q22. How do you find all the indices at which an array holds the value 0? `int ix [$] = arr.find_index with (item == 0);`

Q23. Can you use array methods inside constraints? Yes, the reduction methods (`sum`, `product`, `and`, `or`, `xor`) and `size()` are commonly used in constraints, together with `foreach` and `unique`. Locator and ordering methods cannot be used in constraints; they are applied in `post_randomize()` instead. See Module 5.

Q24. How do you pass an array to a function without copying it? Declare the argument as `ref`, or `const ref` if the function must not modify it. An ordinary `input` argument copies the whole array on every call.

Q25. How do you declare a queue of queues, and an associative array of queues? `int qq [$][$];` and `int aq [string][$];`. Each element of the outer array is itself a queue.

Q26. Is there any difference between `q.delete();` **and** `q = {};`**?** No. Both leave the queue empty.

Q27. How do you get the last element of a dynamic array? Of a queue? `d[d.size() - 1]` and `q[$]`. The `$` shorthand is available only for queues.

Q28. `int a [4] = '{1, 2, 3, 4}; int b [4] = '{4, 3, 2, 1};` **How do you check that** `b` **is the reverse of** `a`**?** Copy and reverse one of them, then compare: `int t [4] = b; t.reverse(); if (t == a) ...`. (In a module-level or static context, split the declaration and the assignment.)

Q29. Which array types can be passed through a module port? Packed arrays and fixed-size unpacked arrays. Dynamic arrays, queues and associative arrays can be arguments of tasks and functions and members of classes, but are not used as synthesizable port types.

Q30. What is the difference between `foreach (m[i, j])` **and nested** `foreach (m[i])` **loops?** The first visits every element of a two-dimensional array in one statement. Nested loops are needed when something must be done between rows (for example printing a newline), or when the inner dimension has a different size per row.

Advanced and scenario-based

Q31. Your scoreboard reports "expected queue not empty" at the end of every test, yet all comparisons passed. What could be wrong? More expected items are pushed than outputs are checked. Typical causes: the monitor misses some DUT outputs (for example it samples on the wrong condition), the DUT legitimately drops or merges some inputs and the predictor does not model that, the test ends before the last outputs emerge (no drain time), or the predictor pushes an expectation for transactions that produce no output (such as those during reset).

Q32. A DUT returns responses in a different order from the requests. How do you change a queue-based scoreboard? Replace the expected queue with an associative array keyed by something that identifies the transaction, such as an ID or address. On each response, look up the key with `exists()`, compare, and `delete()` the entry. At the end of the test `num()` must be 0. If no unique key exists, search the queue with `find_first_index with (item.compare(rsp))` and delete the matching entry.

Q33. A constraint `data.sum() < 100` on `rand bit [7:0] data [4]` produces arrays whose real total is 300 or 560. Why? `sum()` is computed in the 8-bit width of the elements, so it wraps around at 256. A true total of 300 gives an 8-bit sum of 44, which satisfies the constraint. The solver is doing exactly what was written. The fix is to widen each element before adding: `data.sum() with (int'(item)) < 100`. For the same reason a constraint such as `data.sum() == 300` can never be satisfied, because an 8-bit sum cannot reach 300, and randomization fails.

Q34. Why should a memory model use an associative array, and what index type should it have? The address space is far larger than the set of addresses a test touches, and an associative array stores only the written locations. The index should be an unsigned type of the address width, such as `bit [31:0]`, so that traversal is in true address order and high addresses are not treated as negative.

Q35. A testbench becomes slower and uses more memory the longer it runs. Which array-related mistakes would you look for? A queue or associative array that is only ever added to: a history queue that stores every transaction, a scoreboard whose entries are never deleted after comparison, or a monitor that pushes into a queue nobody pops. Also repeated `new[size+1](arr)` reallocation of a large dynamic array, and passing large arrays by value in frequently called functions.

Q36. How would you generate the numbers 0 to 15 in a random order, each exactly once, without `randc`? Fill a queue or array with 0 to 15 and call `shuffle()`. Alternatively randomize an array of 16 elements with `unique {arr}` and each element constrained to the range 0 to 15 (Module 5).

Q37. All ten entries of a queue of transactions show identical field values, equal to the last transaction generated. What is the bug? The same object handle was pushed ten times. Only one object exists and every entry refers to it. A new object must be created for each iteration, or a copy pushed: `q.push_back(txn.copy());`.

Q38. What is the time cost of `q.push_back()`, `q.insert(i, x)`, `aa[key]` and `arr.find with (...)` on N elements? `push_back`, `push_front`, `pop_back` and `pop_front` take constant time. `insert` and `delete(i)` in the middle of a queue must move the following elements, so they are proportional to N. An associative lookup has to search for the key; the cost depends on the implementation and is typically logarithmic in N. `find` is a linear scan, proportional to N.

Q39. How do you compare two queues of transactions when the DUT may deliver them in any order? Sort both by a key that defines a unique order (`q.sort with (item.id)`), then compare element by element using the transaction's `compare()` method. Whole-queue `==` would compare handles, not contents.

Q40. When would `foreach` be the wrong loop? When elements are removed during the loop, when the loop must run backwards or over part of the array, when two arrays must be walked with different indices, or when the loop variable must be changed inside the body.

7.3 Output-Prediction Questions

Work each one out on paper before reading the answer.

P1.

```
module p1;
  int q [$] = {1, 2, 3};
  initial begin
    q.push_front(0);
    q.push_back(4);
    void'(q.pop_front());
    q.insert(2, 9);
    q.delete(0);
    $display("%0d %0d %0d %0d size=%0d", q[0], q[1], q[2], q[3], q.size());
  end
endmodule
```

Answer: `2 9 3 4 size=4`. Steps: `{0,1,2,3}`, `{0,1,2,3,4}`, `{1,2,3,4}`, `{1,2,9,3,4}`, `{2,9,3,4}`.

P2.

```
module p2;
  int d [] = '{1, 2, 3, 4, 5};
  initial begin
    d = new[3](d);
    d = new[5](d);
    $display("%0d %0d %0d %0d %0d", d[0], d[1], d[2], d[3], d[4]);
    d = new[2];
    $display("%0d %0d size=%0d", d[0], d[1], d.size());
  end
endmodule
```

Answer: `1 2 3 0 0` then `0 0 size=2`. Shrinking to 3 drops 4 and 5; growing back to 5 adds two zeros. `new[2]` without a source discards everything.

P3.

```
module p3;
  int aa [int];
  int k, x;
  initial begin
    aa[5] = 1;  aa[-3] = 2;  aa[100] = 3;
    void'(aa.first(k));  $write("%0d ", k);
    void'(aa.next(k));   $display("%0d", k);
    $display("%0d %0d", aa.exists(7), aa.num());
    x = aa[7];
    $display("%0d %0d", x, aa.num());
    aa[7]++;
    $display("%0d %0d", aa[7], aa.num());
  end
endmodule
```

Answer: `-3 5`, then `0 3`, then `0 3`, then `1 4`. Keys are ordered -3, 5, 100. Reading the missing key 7 returns 0 and adds nothing. The increment creates the entry.

P4.

```
module p4;
  bit [3:0] v [] = '{8, 8, 4};
  initial $display("%0d %0d %0d %0d", v.sum(), v.sum() with (int'(item)), v.xor(), v.and());
endmodule
```

Answer: `4 20 4 0`. The 4-bit sum of 20 is 20 mod 16 = 4. With the cast it is 20. `1000 ^ 1000 ^ 0100` = `0100` = 4. `1000 & 1000 & 0100` = 0.

P5.

```
module p5;
  int a [6] = '{5, 3, 8, 3, 9, 1};
  int ix [$], r [$], m [$], u [$];
  initial begin
    ix = a.find_index with (item == 3);
    r  = a.find_last with (item > 4);
    m  = a.min();
    u  = a.unique();
    $display("%0d %0d %0d %0d %0d", ix[0], ix[1], r[0], m[0], u.size());
  end
endmodule
```

Answer: `1 3 9 1 5`. The value 3 is at indices 1 and 3. The last element greater than 4 is 9. The minimum is 1. There are five distinct values.

P6.

```
module p6;
  int a [3:1] = '{10, 20, 30};
  initial begin
    foreach (a[i]) $write("%0d:%0d ", i, a[i]);
    $display("| %0d %0d %0d %0d", $left(a), $right(a), $low(a), $high(a));
  end
endmodule
```

Answer: `3:10 2:20 1:30 | 3 1 1 3`. The pattern fills from the left bound (index 3), and `foreach` iterates from the left bound to the right bound.

P7.

```
module p7;
  logic [3:0][7:0] x [2][5];
  initial $display("%0d %0d %0d %0d %0d %0d", $dimensions(x), $size(x), $size(x, 2),
                   $size(x, 3), $size(x, 4), $bits(x));
endmodule
```

Answer: `4 2 5 4 8 320`. Dimensions 1 and 2 are the unpacked `[2]` and `[5]`; dimensions 3 and 4 are the packed `[3:0]` and `[7:0]`. 2 × 5 × 4 × 8 = 320 bits.

P8.

```
module p8;
  int n, c;
  initial begin
    for (int i = 0; i < 10; i++) begin
      if (i % 2) continue;
      if (i > 6) break;
      n += i;
    end
    do c++; while (c < 0);
    $display("%0d %0d", n, c);
  end
endmodule
```

Answer: `12 1`. Even values 0, 2, 4 and 6 are added; at 8 the loop breaks. The `do`...`while` body runs once although the condition is false.

P9.

```
module p9;
  int q [$] = {1, 2, 3, 4, 5};
  initial begin
    q = q[1:$-1];
    q = {q[$], q[0:$-1]};
    q[$+1] = 7;
    $display("%0d %0d %0d %0d", q[0], q[1], q[2], q[3]);
  end
endmodule
```

Answer: `4 2 3 7`. The slice gives `{2, 3, 4}`. Moving the last element to the front gives `{4, 2, 3}`. Writing to `$+1` appends 7.

P10.

```
module p10;
  int d [] = '{3, 1, 2};
  int e [];
  initial begin
    e = d;
    e.sort();
    d.reverse();
    $display("%0d %0d %0d | %0d %0d %0d", d[0], d[1], d[2], e[0], e[1], e[2]);
    d.rsort();
    $write("%b ", d == e);
    e.reverse();
    $display("%b", d == e);
  end
endmodule
```

Answer: `2 1 3 | 1 2 3`, then `0 1`. `e` is an independent copy, so sorting it leaves `d` alone. `reverse()` only mirrors `{3,1,2}` into `{2,1,3}`. After `rsort`, `d` is `{3,2,1}`, which differs from `{1,2,3}` until `e` is reversed.

P11.

```
module p11;
  class item_c;
    int val;
    function new(int v); val = v; endfunction
  endclass
  item_c a [1], b [1];
  initial begin
    a[0] = new(1);
    b    = a;
    b[0].val = 5;
    b[0] = new(9);
    $display("%0d %0d", a[0].val, b[0].val);
  end
endmodule
```

Answer: `5 9`. After `b = a` both handles refer to one object, so `b[0].val = 5` is seen through `a[0]`. Then `b[0] = new(9)` makes `b[0]` refer to a new object; `a[0]` still refers to the first one.

P12.

```
module p12;
  int n [] = '{5, 10, 15, 20};
  int r [$];
  initial begin
    r = n.find with (item > 8);
    $display("%0d %0d %0d", n.sum() with (item > 8),
             n.sum() with (int'(item > 8)), r.size());
  end
endmodule
```

Answer: `1 3 3`. Three elements exceed 8. The first sum adds three 1-bit ones in a 1-bit result: 3 truncated to one bit is 1. The cast makes the sum 32 bits wide.

7.4 Find-the-Error and Debugging Questions

Compile-time errors

E1.

```
int a [4];
initial $display("%0d", a.size());
```

`size()` is not defined for fixed-size arrays. Use `$size(a)`.

E2.

```
int arr [5] = '{3, 8, 1, 6, 4};
int r;
initial r = arr.find_first with (item > 5);
```

Locator methods return a queue. Declare `int r [$];` and read `r[0]`.

E3.

```
int arr [5] = '{3, 8, 1, 6, 4};
int r [$];
initial r = arr.find(item > 5);
```

The condition must be in a `with` clause: `arr.find with (item > 5)`.

E4.

```
int d [] = new[4];
initial d.push_back(5);
```

`push_back` is a queue method. For a dynamic array use `d = new[5](d); d[4] = 5;`, or declare a queue.

E5.

```
int aa [string];
initial aa.sort();
```

Ordering methods are not available for associative arrays.

E6.

```
int aa [string];
string k;
initial k = aa.first();
```

`first` takes the key variable as a `ref` argument and returns a status: `if (aa.first(k)) ...`.

E7.

```
bit [7:0] m [4];
initial m = 0;
```

A single value cannot be assigned to an unpacked array. Use `m = '{default: 0};`.

E8.

```
int d [];
initial d = new(5);
```

The array constructor uses square brackets: `d = new[5];`. Round brackets belong to the class constructor.

E9.

```
logic [7:0] v = 8'hA5;
int n;
initial n = v.sum();
```

Array methods do not apply to packed vectors. To count ones use `$countones(v)`.

E10.

```
int a [4] = '{1, 2, 3};
```

Three values for four elements. Supply four, or `'{0: 1, 1: 2, 2: 3, default: 0}`.

E11.

```
int d [];
int s [] = d.sort();
```

`sort()` returns nothing. Copy first, then sort: `s = d; s.sort();`.

E12.

```
foreach (arr[i]) begin
  if (arr[i] == 0) i = i + 1;
end
```

The `foreach` loop variable is read-only. Use `continue`, or a `for` loop if the index must be changed.

Run-time bugs (the code compiles)

D1.

```
int d [];
initial begin
  d[0] = 5;
  $display("%0d", d[0]);
end
```

Prints 0. The array was never allocated, so the write is ignored. Add `d = new[1];`.

D2.

```
bit [7:0] payload [] = '{200, 200, 200};
int total;
initial total = payload.sum();
```

`total` is 88, not 600: the sum is computed in 8 bits (600 mod 256). Use `payload.sum() with (int'(item))`.

D3.

```
for (int i = 0; i <= q.size(); i++) $display("%0d", q[i]);
```

One pass too many: `q[q.size()]` does not exist. Use `<`.

D4.

```
int count [string];
initial if (count["ADD"] == 0) $display("ADD never seen");
```

A missing key and a stored 0 cannot be distinguished, and reading a missing key produces a warning. Use `if (!count.exists("ADD"))`.

D5.

```
foreach (q[i]) if (q[i] < 0) q.delete(i);
```

Deleting while iterating forward skips the element after each deleted one, and the behaviour of `foreach` when the size changes is undefined. Loop backwards with `for`, or `q = q.find with (item >= 0);`.

D6.

```
d = new[d.size() + 1];
d[d.size() - 1] = new_value;
```

The existing contents are lost because the source array was not passed. Write `d = new[d.size() + 1](d);`.

D7.

```
expected = exp_q.pop_front();
if (actual !== expected) $error("mismatch");
```

If `exp_q` is empty, `pop_front()` returns the default value and the comparison is made against garbage (or passes by coincidence when the DUT outputs 0). Check `exp_q.size()` first and report an unexpected output.

D8.

```
alu_txn txn = new();
repeat (5) begin
  void'(txn.randomize());
  q.push_back(txn);
end
```

All five queue entries are the same handle, so they all show the last random values. Create the object inside the loop, or push `txn.copy()`.

D9.

```
for (int unsigned i = arr.size() - 1; i >= 0; i--) $display("%0d", arr[i]);
```

Infinite loop: an unsigned variable is never negative. Use `int`.

D10.

```
logic [7:0] mem [int];
...
foreach (mem[addr]) $display("%h", addr);     // addresses above 7FFF_FFFF print first
```

The signed index type makes addresses with the top bit set negative, so they sort before 0. Declare the index as `bit [31:0]`.

D11.

```
int m [2][3];
initial foreach (m[i]) foreach (m[j]) m[i][j] = 7;
```

Column 2 is never written. The inner loop iterates over the first dimension again, so `j` only takes the values 0 and 1. Write `foreach (m[i, j]) m[i][j] = 7;`.

7.5 Coding Exercises with Solutions

Attempt each exercise before reading the solution. The solutions are also in `examples/module-02/`.

Exercise 1: rotate an array

Rotate `'{1, 2, 3, 4, 5}` left by `k` = 2 positions, once with a fixed-size array and index arithmetic, once with a queue.

ex01_rotate.sv

```
module ex01_rotate;

  int arr [5] = '{1, 2, 3, 4, 5};
  int rot [5];
  int q [$];
  int k = 2;

  initial begin
    // 1. index arithmetic: element i of the result comes from index (i + k) modulo size
    foreach (arr[i]) rot[i] = arr[(i + k) % $size(arr)];
    $display("fixed array : %p", rot);

    // 2. queue: move the front element to the back, k times
    q = arr;
    repeat (k) q.push_back(q.pop_front());
    $display("queue       : %p", q);
  end

endmodule
```

EXPECTED OUTPUT

```
fixed array : '{3, 4, 5, 1, 2}
queue       : '{3, 4, 5, 1, 2}
```

Exercise 2: second-largest value

Find the second-largest distinct value of `'{12, 45, 7, 45, 30}` with a single loop, then with array methods. Assume the values are not negative.

ex02_second_largest.sv

```
module ex02_second_largest;

  int a [5] = '{12, 45, 7, 45, 30};
  int first, second;
  int r [$];

  initial begin
    first  = -1;
    second = -1;
    foreach (a[i]) begin
      if (a[i] > first) begin
        second = first;
        first  = a[i];
      end
      else if (a[i] > second && a[i] != first)
        second = a[i];
    end
    $display("loop    : largest = %0d, second largest = %0d", first, second);

    r = a.unique();
    r.rsort();
    $display("methods : largest = %0d, second largest = %0d", r[0], r[1]);
  end

endmodule
```

EXPECTED OUTPUT

```
loop    : largest = 45, second largest = 30
methods : largest = 45, second largest = 30
```

Trace of the loop: 12 becomes `first`. 45 pushes 12 down to `second`. 7 changes nothing. The second 45 equals `first`, so it is ignored (the `!= first` test). 30 is greater than `second` (12) and replaces it.

Exercise 3: remove duplicates, keeping the original order

ex03_remove_duplicates.sv

```
module ex03_remove_duplicates;

  int in_q  [$] = {3, 1, 3, 2, 1, 5};
  int out_q [$];
  bit seen  [int];                    // associative array used as a set

  initial begin
    foreach (in_q[i])
      if (!seen.exists(in_q[i])) begin
        seen[in_q[i]] = 1;
        out_q.push_back(in_q[i]);
      end
    $display("%p", out_q);
  end

endmodule
```

EXPECTED OUTPUT

```
'{3, 1, 2, 5}
```

An associative array whose values do not matter acts as a set: `exists()` answers "have I seen this value before?". `unique()` would also remove duplicates, but the standard does not promise the order of its result.

Exercise 4: frequency count

Count how often each operation name appears and report the most frequent one.

ex04_frequency.sv

```
module ex04_frequency;

  string ops [6] = '{"ADD", "SUB", "ADD", "XOR", "ADD", "SUB"};
  int    freq [string];
  string best;

  initial begin
    foreach (ops[i]) freq[ops[i]]++;

    foreach (freq[k]) $display("%s = %0d", k, freq[k]);

    best = "";
    foreach (freq[k])
      if (best == "" || freq[k] > freq[best]) best = k;
    $display("most frequent: %s (%0d times)", best, freq[best]);
  end

endmodule
```

EXPECTED OUTPUT

```
ADD = 3
SUB = 2
XOR = 1
most frequent: ADD (3 times)
```

Exercise 5: reference model of a FIFO

Model a FIFO of depth 4 with a queue. Report overflow on a write to a full FIFO and underflow on a read from an empty FIFO. This is the reference model a scoreboard would use for the `sync_fifo` DUT.

ex05_fifo_model.sv

```
module ex05_fifo_model;

  localparam int DEPTH = 4;
  bit [7:0] fifo_q [$];

  function automatic void fifo_write(bit [7:0] data);
    if (fifo_q.size() == DEPTH) begin
      $display("write %h : FULL, data dropped", data);
      return;
    end
    fifo_q.push_back(data);
    $display("write %h : ok   (count %0d)", data, fifo_q.size());
  endfunction

  function automatic void fifo_read();
    bit [7:0] data;
    if (fifo_q.size() == 0) begin
      $display("read     : EMPTY");
      return;
    end
    data = fifo_q.pop_front();
    $display("read  %h : ok   (count %0d)", data, fifo_q.size());
  endfunction

  initial begin
    fifo_write(8'h11);
    fifo_write(8'h22);
    fifo_write(8'h33);
    fifo_write(8'h44);
    fifo_write(8'h55);
    fifo_read();
    fifo_read();
    fifo_write(8'h66);
    repeat (4) fifo_read();
  end

endmodule
```

EXPECTED OUTPUT

```
write 11 : ok   (count 1)
write 22 : ok   (count 2)
write 33 : ok   (count 3)
write 44 : ok   (count 4)
write 55 : FULL, data dropped
read  11 : ok   (count 3)
read  22 : ok   (count 2)
write 66 : ok   (count 3)
read  33 : ok   (count 2)
read  44 : ok   (count 1)
read  66 : ok   (count 0)
read     : EMPTY
```

`size() == DEPTH` is the model's `full` flag and `size() == 0` its `empty` flag. Data leaves in the order it entered.

Exercise 6: transpose a matrix

ex06_transpose.sv

```
module ex06_transpose;

  int m [2][3] = '{ '{1, 2, 3}, '{4, 5, 6} };
  int t [3][2];

  initial begin
    foreach (m[i, j]) t[j][i] = m[i][j];
    foreach (t[i]) $display("%0d %0d", t[i][0], t[i][1]);
  end

endmodule
```

EXPECTED OUTPUT

```
1 4
2 5
3 6
```

Exercise 7: one-line answers with array methods

For `'{4, 9, 2, 9, 7, 2, 5}` compute, without writing a loop: the sum of the even values, the number of values above 5, the largest value and its first index, the second-smallest distinct value, whether all values are below 10, and whether 7 is present.

ex07_methods.sv

```
module ex07_methods;

  int d [] = '{4, 9, 2, 9, 7, 2, 5};
  int r [$], ix [$];

  initial begin
    $display("sum of even values       = %0d", d.sum() with (item % 2 == 0 ? item : 0));
    $display("number of values above 5 = %0d", d.sum() with (int'(item > 5)));

    r  = d.max();
    ix = d.find_first_index with (item == r[0]);
    $display("largest = %0d at index %0d", r[0], ix[0]);

    r = d.unique();
    r.sort();
    $display("second-smallest distinct = %0d", r[1]);

    r = d.find with (item >= 10);
    $display("all values below 10      : %s", (r.size() == 0) ? "yes" : "no");
    $display("contains 7               : %s", (7 inside {d}) ? "yes" : "no");
  end

endmodule
```

EXPECTED OUTPUT

```
sum of even values       = 8
number of values above 5 = 3
largest = 9 at index 1
second-smallest distinct = 4
all values below 10      : yes
contains 7               : yes
```

Exercise 8: find the missing number

An array holds the numbers 1 to N with exactly one missing. Find it.

ex08_missing_number.sv

```
module ex08_missing_number;

  int arr [] = '{1, 2, 4, 5, 6};
  int n, missing;

  initial begin
    n       = arr.size() + 1;                // the complete set would have n numbers
    missing = n * (n + 1) / 2 - arr.sum();   // sum of 1..n minus the actual sum
    $display("N = %0d, missing number = %0d", n, missing);
  end

endmodule
```

EXPECTED OUTPUT

```
N = 6, missing number = 3
```

1 + 2 + ... + 6 = 21, and the array sums to 18. The elements are `int`, so `sum()` has no overflow problem here.

Exercise 9: are two arrays permutations of each other?

ex09_permutation.sv

```
module ex09_permutation;

  function automatic bit is_permutation(int x [], int y []);
    if (x.size() != y.size()) return 0;
    x.sort();                  // x and y are copies: the caller's arrays are not changed
    y.sort();
    return (x == y);
  endfunction

  int a [] = '{3, 1, 2};
  int b [] = '{2, 3, 1};
  int c [] = '{3, 1, 1};

  initial begin
    $display("a and b : %b", is_permutation(a, b));
    $display("a and c : %b", is_permutation(a, c));
    $display("a is still %p", a);
  end

endmodule
```

EXPECTED OUTPUT

```
a and b : 1
a and c : 0
a is still '{3, 1, 2}
```

Arrays passed as ordinary arguments are copied, so sorting inside the function does not disturb the caller. Sorted copies of two permutations are identical.

Exercise 10: row and column totals

ex10_row_col_sums.sv

```
module ex10_row_col_sums;

  int m [2][3] = '{ '{1, 2, 3}, '{4, 5, 6} };
  int col;

  initial begin
    foreach (m[i]) $display("row %0d total = %0d", i, m[i].sum());

    for (int j = 0; j < $size(m, 2); j++) begin
      col = 0;
      for (int i = 0; i < $size(m, 1); i++) col += m[i][j];
      $display("column %0d total = %0d", j, col);
    end
  end

endmodule
```

EXPECTED OUTPUT

```
row 0 total = 6
row 1 total = 15
column 0 total = 5
column 1 total = 7
column 2 total = 9
```

A row `m[i]` is a one-dimensional array, so `sum()` can be called on it. A column is not a contiguous sub-array, so it needs a loop.

Further practice (no solutions given)

1. Write a function that returns 1 if an array of `int` is a palindrome (reads the same in both directions).
2. Merge two sorted queues into one sorted queue without calling `sort()`.
3. Given `bit [7:0] data [$]`, split it into two queues: values with even parity and values with odd parity.
4. Implement a stack with `push`, `pop` and `peek` functions on a queue, with underflow reporting.
5. Model a 64 KB memory with an associative array. Add a function that prints every written address range as `start-end` when consecutive addresses have been written.
6. Given an array of 9-bit ALU results, print the three largest values and their indices.
7. Count the number of distinct values in an array without using `unique()`.
8. Fill a 4 × 4 two-dimensional array so that element `[i][j]` is `i * 4 + j`, then print its two diagonals.
9. Predict, then run: `int q [$] = {1, 2, 3}; foreach (q[i]) q.push_front(q[i]);` Explain why this loop is dangerous.
10. Write the bubble sort of Section 6.5 so that it stops early when a complete pass makes no swap.

7.6 Concepts You Must Understand Before Moving Forward

Do not start Module 3 until you can answer each of these without looking at the notes.

Array kinds

- [ ] I can declare a fixed-size array, a dynamic array, a queue and an associative array, and say when each is used in a testbench.

- [ ] I know which arrays are synthesizable.
- [ ] I can explain `$size` versus `size()`, and `$left`/`$right` versus `$low`/`$high`.
- [ ] I can number the dimensions of a mixed packed/unpacked array.

Dynamic arrays and queues

- [ ] I can explain the difference between `new[n]` and `new[n](d)`.
- [ ] I know that a dynamic array never grows on assignment.
- [ ] I can write a FIFO and a stack with queue methods.
- [ ] I know what `pop_front()` returns on an empty queue and why the size must be checked.
- [ ] I can rewrite each queue method as a slice/concatenation expression.

Associative arrays

- [ ] I can explain why a memory model uses an associative array.
- [ ] I can traverse an associative array with `first`/`next` and with `foreach`, and I know the order.
- [ ] I know what reading a missing key does and why `exists()` is required.

Array methods

- [ ] I know that every locator method returns a queue and that `find...` needs a `with` clause.
- [ ] I can explain why `sum()` of a byte array can give a wrong total, and fix it.
- [ ] I can count matching elements with `sum()` and with `find`.
- [ ] I can explain `rsort()` versus `reverse()`.
- [ ] I can sort an array of structs or transactions by a field.

Copying and loops

- [ ] I can explain why `b = a` gives independent arrays, except that arrays of class handles share objects.
- [ ] I can write a deep copy of an array of handles.
- [ ] I can explain `while` versus `do`...`while` and `for` versus `foreach`.
- [ ] I can iterate over a two-dimensional array in both ways.
- [ ] I know how to delete elements from a queue safely inside a loop.

7.7 What Comes Next

Module 3 covers tasks and functions: the differences between them, argument directions (`input`, `output`, `inout`, `ref`), pass by value and pass by reference, default arguments, `automatic` and `static` lifetime, `void` functions, recursion, and timing control inside tasks. It uses the arrays of this module heavily, in particular to show why large arrays are passed by `ref`, and it explains the functions that the examples in this module used informally.